

BASS PAGO · REVOPS

RevenueOps Cockpit

Documentação técnica, funcional e executiva do sistema interno de operação e gestão da Bass Pago.

Versão do documento	1.0
Data	10 de setembro de 2026
Commit de referência	4890630
Branch	claude/setup-deploy-cockpit-SjSte
Ambiente	Production
Banco	PostgreSQL 17.6 (Supabase)

Sumário

1 O que é o RevenueOps Cockpit

- 1.1 O problema central que ele resolve
- 1.2 O que o Cockpit NÃO é
- 1.3 Quem usa e o que cada um extrai
- 1.4 As quatro camadas de leitura

2 Estado real do sistema

3 História e evolução da arquitetura

- 3.1 Origem — um CRM genérico
- 3.2 Transformação em sistema de instituição de pagamentos
- 3.3 A refatoração dos KPIs — a decisão mais estruturante
- 3.4 Redesign institucional
- 3.5 Volumetria por cliente — migration v10
- 3.6 Pipeline multi-funil — migration v11
- 3.7 Consolidação e novos ambientes — migrations v12 a v14
- 3.8 v15 — alinhamento do enum MetaTipo
- 3.9 Consolidação de duas frentes de trabalho
- 3.10 Infraestrutura

4 Decisões de arquitetura e seus porquês

- 4.1 Por que o CRM não tem entidade própria
- 4.2 Por que `PipelineMovimentacao` existe, se já há `Auditoria`
- 4.3 Por que as Automações não usam event bus
- 4.4 Por que a automação roda depois da transação
- 4.5 Por que os formulários guardam JSON versionado
- 4.6 Por que arquivos ficam em Storage privado, e não no PostgreSQL
- 4.7 Por que a senha de certificado usa AES-256-GCM, e não hash
- 4.8 Por que o Float é derivado e nunca lançado
- 4.9 Por que não existe TPV por cliente
- 4.10 Por que não existe reconciliação

5 **Usuários, papéis e permissões**

- 5.1 Os quatro papéis
- 5.2 O mecanismo de permissões
- 5.3 Matriz das chaves sensíveis
- 5.4 Alçada por funil
- 5.5 Autorização é sempre server-side

6 **Arquitetura técnica**

- 6.1 Stack
- 6.2 O fluxo de uma requisição autenticada
- 6.3 O fluxo de um arquivo
- 6.4 Separação entre domínio puro e acesso a dados
- 6.5 O registro de módulos

7 **Catálogo do banco de dados**

- 7.1 Visão conceitual do domínio
- 7.2 Classificação das 35 entidades
- 7.3 Entidades centrais
- 7.4 Fundação de indicadores
- 7.5 Pipeline — 4 modelos
- 7.6 Certificados — 3 modelos, dois conceitos
- 7.7 Demais modelos
- 7.8 Enums que carregam regra de negócio

8 **Dashboard (Cockpit)**

- 8.1 Os KPIs e suas origens
- 8.2 As 5 linhas de faturamento
- 8.3 Metas e Volumetria no Dashboard
- 8.4 Painel de Insights
- 8.5 Alertas — diferentes de Notificações

9 **Float — o indicador mais sutil do sistema**

- 9.1 O conceito
- 9.2 O que é "saldo que dorme"

- 9.3 Exemplo numérico
 - 9.4 Gaps
 - 9.5 Multiplicador versionado
 - 9.6 Por que é derivado e nunca lançado
-

10 **Volumetria mínima por cliente**

- 10.1 O conceito
 - 10.2 Volumetria $\neq$ TPV
 - 10.3 Contrato individual $\times$ consolidado
 - 10.4 Status de um contrato
 - 10.5 Regra de não sobreposição
 - 10.6 O problema empresarial resolvido
-

11 **Carteira Comercial**

- 11.1 O que a tela mostra
 - 11.2 Gestor e o vínculo legado
 - 11.3 O indicador dos últimos 5 dias
 - 11.4 Por que esse indicador existe — e o que ele NÃO é
 - 11.5 Expectativa e potencial
-

12 **CRM**

- 12.1 O cadastro
 - 12.2 Por que o CRM não tem entidade própria
 - 12.3 As métricas, uma a uma
 - 12.4 Escopo de leitura
-

13 **Pipeline multi-funil**

- 13.1 Os três funis semeados
 - 13.2 Vocabulário
 - 13.3 Mover $\times$ Transferir
 - 13.4 Uma transferência, passo a passo
 - 13.5 Administração de funis
 - 13.6 Permissões, uma vez mais
-

14 **Notificações**

- 14.1 O que é — e o que não é
 - 14.2 O modelo
 - 14.3 De onde nascem, hoje
 - 14.4 Interface
 - 14.5 Segurança
-

15 Documentos

- 15.1 Categorias e formatos
 - 15.2 Upload
 - 15.3 Validação
 - 15.4 Download
 - 15.5 Inativação, não exclusão
 - 15.6 Por que os bytes não ficam no PostgreSQL
-

16 Gestão de Certificados

- 16.1 Dois conceitos que não se misturam
 - 16.2 Numeração relativa à versão
 - 16.3 Criação de uma versão
 - 16.4 Registro de um envio
 - 16.5 Cancelamento
 - 16.6 Criptografia — AES-256-GCM
 - 16.7 O risco operacional que isso resolve
-

17 Compliance

- 17.1 O modelo
 - 17.2 Máquina de estados
 - 17.3 Prazo
 - 17.4 Fluxo operacional
 - 17.5 Histórico
 - 17.6 O problema resolvido
-

18 Automações

- 18.1 A arquitetura no-code
- 18.2 Escopo e condição

- 18.3 Por que não existe event bus
 - 18.4 Por que roda depois da transação
 - 18.5 ``AutomacaoExecucao``
 - 18.6 As ações, em detalhe
 - 18.7 Validação na gravação
-

19 Formulários

- 19.1 Os cinco modelos
 - 19.2 Os 24 tipos de campo
 - 19.3 Propriedades por campo
 - 19.4 O construtor
 - 19.5 Versionamento e imutabilidade
 - 19.6 O link público ``/f/[token]``
 - 19.7 Validação da resposta
 - 19.8 Anexos
 - 19.9 Paineis
-

20 Auditoria

- 20.1 O modelo
 - 20.2 O que é auditado hoje
 - 20.3 Os dois casos mais sensíveis
 - 20.4 O que NÃO é registrado
 - 20.5 Três tabelas de histórico coexistem com a Auditoria
-

21 Segurança

- 21.1 Autenticação
- 21.2 Autorização em três camadas
- 21.3 Segredos
- 21.4 Storage
- 21.5 Criptografia
- 21.6 Link público de formulário
- 21.7 Exposição de dados
- 21.8 Limitação conhecida

22 Design System

- 22.1 Filosofia
- 22.2 Tipografia
- 22.3 Tokens
- 22.4 Light / Dark Mode
- 22.5 Componentes
- 22.6 Formulários e responsividade
- 22.7 Marca

23 Testes

- 23.1 Estratégia
- 23.2 Cobertura por área
- 23.3 Os testes que pegam o que revisão de código não pega
- 23.4 O que a suíte NÃO cobre

24 Deploy e infraestrutura

- 24.1 Ambientes
- 24.2 O processo
- 24.3 Migrations
- 24.4 Conexões
- 24.5 Storage
- 24.6 Estado de Production no momento deste documento

25 Manutenção — guia para quem vier depois

- 25.1 Alterar o schema
- 25.2 Adicionar uma permissão
- 25.3 Criar uma rota de API
- 25.4 Adicionar uma automação
- 25.5 Adicionar um tipo de campo de formulário
- 25.6 Adicionar um módulo
- 25.7 Antes de qualquer publicação
- 25.8 Invariantes do produto

26 Limitações e débitos técnicos

27 Roadmap sugerido

28 Glossário

29 Diagramas consolidados

- 29.1 Arquitetura técnica
- 29.2 Fluxo de dados dos indicadores
- 29.3 Pipeline e transferência
- 29.4 Automações
- 29.5 Documentos e Storage
- 29.6 Certificados
- 29.7 Formulários
- 29.8 Permissões

RevenueOps Cockpit — Bass Pago RevOps

Documentação Técnica, Funcional e Executiva

Versão do documento	1.0
Data	10 de setembro de 2026
Commit de referência	4890630 (= 6fed672 + migration v15)
Branch	claude/setup-deploy-cockpit-SjSte
Ambiente documentado	Production — https://revenueops-cockpit.vercel.app
Base de dados	Supabase gyvs...zchd · PostgreSQL 17.6

Este documento descreve o sistema **como ele está no código**, não como foi idealizado. Onde algo está parcialmente implementado, isso é dito explicitamente. Onde uma funcionalidade existe apenas como estrutura de dados sem interface, isso também é dito.

1 O que é o RevenueOps Cockpit

O RevenueOps Cockpit é o **sistema interno de operação e gestão da Bass Pago**, uma instituição de pagamentos. Ele concentra, num único lugar, a informação que antes vivia espalhada entre planilhas, conversas e a cabeça de cada pessoa: quanto a operação processou, quanto faturou, quais clientes estão na carteira, em que ponto cada negócio está, o que o Compliance precisa resolver e quais documentos existem de cada cliente.

1.1 O problema central que ele resolve

Numa instituição de pagamentos, três perguntas precisam ter **uma única resposta**, e não uma por departamento:

1. **Quanto processamos e quanto ganhamos com isso?**
2. **Em que estágio está cada cliente — do primeiro contato até a sustentação?**
3. **O que está pendente, com quem, e desde quando?**

Antes do Cockpit, cada área respondia isso do seu jeito. O comercial tinha uma planilha de pipeline; o financeiro, outra de receita; a operação, uma terceira de incidentes. Os números não fechavam entre si porque não vinham da mesma fonte.

O Cockpit resolve isso com uma regra que atravessa todo o sistema:

Cada indicador tem UMA fonte oficial. Quando não há dado, o valor é `null` e a tela mostra estado vazio — nunca zero.

Essa regra está escrita no topo de `lib/kpi.ts` e é o princípio de design mais importante do produto. Um zero falso é pior que um vazio honesto, porque um zero parece uma medição.

1.2 O que o Cockpit NÃO é

Delimitar isso importa tanto quanto descrever o que ele faz:

- **Não é um sistema de processamento.** Ele não move dinheiro, não autoriza transação, não fala com adquirente ou arranjo de pagamento. Ele **registra e interpreta** o que o processamento produziu.
- **Não é um ERP nem um sistema contábil.** Não emite nota, não faz contabilidade, não fecha balanço.
- **Não faz reconciliação.** Não existe conciliação entre extrato e transação — foi uma decisão explícita de escopo.
- **Não mede TPV por cliente.** O volume financeiro é registrado no agregado diário da empresa, nunca quebrado por cliente. Ver §11.3.
- **Não é um produto para o cliente final.** Todo usuário é interno, com uma única exceção: o link público de formulário (§19.6).

1.3 Quem usa e o que cada um extrai

Perfil	O que faz no sistema	Qual decisão toma
Conselho / Diretoria	Lê o Cockpit e o Conselho	Alocação de capital, metas, avaliação de resultado
Gestor de carteira	Carteira, CRM, Pipeline, Follow-up	Onde investir tempo comercial, qual cliente está esfriando
Operação	Incidentes, Tarefas, Onboarding, Compliance	O que precisa de ação hoje, o que está fora do prazo
Comercial	Leads, Pipeline de Vendas	Qual negócio avançar, qual está parado
Administração	Usuários, Parâmetros, Funis, Automações, Auditoria	Como o sistema se comporta e quem acessa o quê

1.4 As quatro camadas de leitura

O produto é organizado em quatro alturas de decisão — e é útil entender qual tela serve a qual altura:

- **Operação** (Incidentes, Tarefas, Compliance, Documentos, Certificados): responde "*o que preciso fazer agora*". Granularidade de item.
- **Comercial** (Leads, Pipeline, CRM, Carteira, Follow-up): responde "*onde está cada negócio e cada cliente*". Granularidade de relacionamento.
- **Gestão** (Cockpit, Metas, Volumetria, Relatórios, Métricas Op.): responde "*como estamos indo contra o planejado*". Granularidade de período.
- **Conselho** (Conselho, Relatórios executivos): responde "*a tese está se confirmando*". Granularidade de tendência.

2 Estado real do sistema

Números apurados diretamente do repositório e do banco de Production no commit de referência:

Dimensão	Quantidade
Modelos Prisma	35
Enums Prisma	30
Tabelas no banco de Production	41 (35 do schema + 6 legadas)
Enums no banco de Production	31 (30 + <code>PedidoStatus</code> legado)
Rotas de API (<code>route.ts</code>)	67
Handlers HTTP	116
Páginas	34
Componentes React	29
Módulos de domínio em <code>lib/</code>	26
Migrations SQL versionadas	14 (v1...v15, com saltos históricos)
Testes automatizados	104, todos passando
Chaves de permissão	42
Linhas em <code>app/</code> + <code>lib/</code> + <code>components/</code>	~21.300
Chaves estrangeiras em Production	67
Índices em Production	100 (64 únicos)

3 História e evolução da arquitetura

O repositório tem 81 commits. A evolução se deu em fases claras, e entender por que cada mudança aconteceu explica a forma atual do sistema.

3.1 Origem — um CRM genérico

O primeiro commit ([89cb739](#) , "build complete RevenueOps Cockpit from scratch") criou um CRM convencional: `Lead` , `Deal` , `Activity` , `User` . Estrutura de funil de vendas clássica, com `DealStage` como enum fixo.

3.2 Transformação em sistema de instituição de pagamentos

[9d41d5c](#) ("transform CRM into RevenueOps Cockpit for payment institution") trouxe o domínio real: `Cliente` , `ContaReceber` , `Incidente` , `Meta` , `Parametro` . O sistema deixou de ser sobre vender e passou a ser sobre **operar**.

Nessa fase nasceram tabelas que hoje são legado: `Forecast` , `ForecastGeral` , `IncidenteCliente` , `PedidoCobavel` , `Processamento` , `ReceitaRealizada` . Elas continuam no banco, vazias, preservadas (§26).

3.3 A refatoração dos KPIs — a decisão mais estruturante

Em algum ponto o sistema acumulou **36 fallbacks silenciosos**: quando um dado faltava, a tela mostrava zero. Isso produzia relatórios que pareciam medições e não eram.

A refatoração criou `lib/kpi.ts` como **registro central de indicadores**, com uma fonte oficial declarada para cada um:

```
LancamentoDiario → TPV, Receita Tarifária, Saldo em Conta, Transações, MEDs
FloatConfig      → multiplicador do Float (derivado, nunca lançado)
Cliente          → MRR, WL Ativos, Contas Ativas, BaaS Ativos
ContaReceber     → Setup e Serviços
Meta             → objetivos (somente o alvo)
```

E substituiu os fallbacks pela regra do `null` . **Esta é a decisão arquitetural que mais define o produto hoje.**

3.4 Redesign institucional

[26cb060](#) ("redesign institucional do cockpit") trocou uma UI genérica por um design system próprio: tokens de cor, escala tipográfica, componentes primitivos (`Panel` , `Badge` , `DataTable` , `StatTile` , `EmptyState`). Ver §22.

3.5 Volumetria por cliente — migration v10

A volumetria mínima era **um número geral da empresa por mês**. Isso estava errado para o produto: volumetria mínima é uma **cláusula contratual de um cliente específico**.

A migration v10 evoluiu o modelo existente em vez de criar outro: `VolumetriaMinima` ganhou `clienteId`, `vigenciaFim` e `ativo`; a coluna `periodo` passou a ser o início da vigência; o índice único de `periodo` deu lugar a `(clienteId, periodo)`.

Por que evoluir e não criar novo: um modelo novo duplicaria o conceito e deixaria a tabela antiga órfã. Evoluindo, as linhas antigas viram "contrato geral legado" (`clienteId` nulo, somente leitura) e os alertas de meses já fechados continuam produzindo o mesmo resultado.

3.6 Pipeline multi-funil — migration v11

O Pipeline era um Kanban com colunas **fixas no código** — o array `STAGES` duplicado em dois arquivos. Virou uma estrutura configurável: `PipelineFunil`, `PipelineEtapa`, `PipelinePermissao`, `PipelineMovimentacao`.

Decisão central: o card continua sendo o Deal. Ele ganhou `funilId`, `etapaId` e `clienteId`. A alternativa — um `PipelineCard` polimórfico — custaria um modelo novo, migração de todos os deals e reescrita das integrações, sem entregar nada a mais. Ver §13.

3.7 Consolidação e novos ambientes — migrations v12 a v14

Três migrations agrupadas por acoplamento, não por módulo:

- **v12 (Fundação):** todos os enums novos de uma vez, role `GESTOR`, `Cliente.gestorId`, `Notificacao`, `Documento`, `ClienteDiaMovimento`, `LeadComentario`.
- **v13 (Certificados + Compliance):** `CertificadoVersao`, `Certificado`, `CertificadoEnvio`, `PendenciaCompliance`, `PendenciaEvento`.
- **v14 (Automações + Formulários):** `Automacao`, `AutomacaoExecucao` e os cinco modelos de formulário.

Por que 3 e não 10: criar um enum na v14 que a v12 já poderia ter criado é exatamente a fragmentação que um plano consolidado evita.

3.8 v15 — alinhamento do enum MetaTipo

O `schema.prisma` declarava três valores legados de `MetaTipo` (`RECEITA`, `MRR`, `CLIENTES_ATIVOS`) que a limpeza do banco havia removido. Prisma Client e banco discordavam sobre um tipo — bug latente que só apareceria em produção. A v15 é puramente `ALTER TYPE ... ADD VALUE IF NOT EXISTS`.

3.9 Consolidação de duas frentes de trabalho

O commit `6fed672` ("consolidate product structure and design system") uniu duas linhas de trabalho paralelas: a de **estrutura de produto** (módulos, domínio, migrations) e a de **design system** (Light/Dark Mode, tipografia, componentes). O documento atual descreve o resultado consolidado.

3.10 Infraestrutura

- **Supabase** como PostgreSQL gerenciado, desde o início.
- **Vercel** como plataforma de deploy, com Preview e Production.
- **Supabase Storage** introduzido junto com o módulo de Documentos, em bucket privado.

4 Decisões de arquitetura e seus porquês

Esta seção existe porque a forma do sistema só faz sentido junto com as razões.

4.1 Por que o CRM não tem entidade própria

O CRM é **analítica pura** sobre dados que já existem. Ele lê `Deal` e `PipelineMovimentacao` e deriva tempo médio por etapa, conversão, ciclo e gargalos.

Criar um `CrmDeal` ou uma tabela de métricas agregadas significaria manter duas verdades sobre o mesmo fato — e elas divergiriam no primeiro bug de sincronização. `lib/crm.ts` não tem um único `INSERT`.

4.2 Por que `PipelineMovimentacao` existe, se já há `Auditoria`

`Auditoria.detalhes` é **texto livre**. Serve para responder "quem fez o quê", mas não permite montar uma linha do tempo com origem e destino, nem filtrar "todos os cards que saíram de Negociação em agosto".

`PipelineMovimentacao` é estruturada: `funilOrigemId`, `etapaOrigemId`, `funilDestinoId`, `etapaDestinoId`, `userId`, `createdAt`. É dela que sai toda a analítica do CRM. As duas coexistem: movimentação para a timeline do card, auditoria para a trilha administrativa global.

O mesmo raciocínio justifica `PendenciaEvento` e `AutomacaoExecucao`.

4.3 Por que as Automações não usam event bus

Porque **o ponto único já existia**. `registrarMovimentacao()`, em `lib/pipeline-db.ts`, é a única função por onde passa qualquer mudança de card — criação, movimento de etapa e transferência de funil. Os gatilhos se penduram nela.

Um event bus acrescentaria fila, entrega garantida, ordenação e observabilidade própria para resolver um problema que quatro chamadas de função já resolvem.

4.4 Por que a automação roda depois da transação

Uma automação com defeito **não pode desfazer uma movimentação de card que já aconteceu**. O commit acontece primeiro; o disparo vem depois, fora da transação, e toda falha vira uma linha em `AutomacaoExecucao` em vez de uma exceção que sobe.

`dispararAutomacoes()` nunca lança — nem quando a própria listagem de automações falha.

4.5 Por que os formulários guardam JSON versionado

A estrutura de um formulário mora num único campo `FormularioVersao.definicao` (JSONB), em vez de tabelas `Campo`, `Opcao`, `Secao`.

Três razões:

1. **Imutabilidade de graça.** Publicada e respondida, a versão vira snapshot. Editar cria a próxima. Nenhuma alteração de hoje muda o significado do que foi respondido ontem.
2. **Menos superfície.** Some três tabelas e o join correspondente.
3. **O usuário nunca vê JSON.** O construtor visual gera e interpreta.

O custo consciente: relatório sobre respostas exige trabalhar com JSONB.

4.6 Por que arquivos ficam em Storage privado, e não no PostgreSQL

Bytes em banco relacional inflam backup, complicam replicação e transformam cada download numa consulta. O PostgreSQL guarda **apenas a chave** (`Documento.storageKey`); os bytes vivem no bucket privado `cliente-arquivos`.

O bucket **nunca** é público: todo download passa por uma rota autenticada que checa permissão, grava auditoria e emite uma *signed URL* de 60 segundos.

4.7 Por que a senha de certificado usa AES-256-GCM, e não hash

Senha de certificado **precisa ser lida de volta** para ser entregue ao cliente. Hash não serve — é irreversível por construção.

AES-256-GCM foi escolhido porque, além de cifrar, **autentica**: um registro adulterado no banco falha ao decifrar, em vez de devolver lixo silenciosamente.

4.8 Por que o Float é derivado e nunca lançado

Float é o rendimento do dinheiro que dorme em conta. Se alguém pudesse digitá-lo, ele deixaria de ser uma medição e passaria a ser uma opinião.

Ele é calculado a partir do saldo diário (`LancamentoDiario.saldoEmConta`) e do multiplicador vigente (`FloatConfig`). Ver §9.

4.9 Por que não existe TPV por cliente

Decisão de produto, não limitação técnica. O TPV é registrado no agregado diário da empresa. Quebrar por cliente exigiria uma fonte de dados que o Cockpit não possui, e produziria um número que pareceria preciso sem ser.

O que existe por cliente é a **expectativa** (`Cliente.tpvEsperado`) e um **indicador binário de movimentação diária** (§11.3) — nunca valor realizado.

4.10 Por que não existe reconciliação

Fora de escopo por decisão explícita. Conciliar extrato contra transação exige integração com arranjos de pagamento e uma máquina de matching — outro produto.

5 Usuários, papéis e permissões

5.1 Os quatro papéis

O enum `Role` tem quatro valores. Os rótulos de interface vêm de `ROLE_LABELS` em `lib/utils.ts`:

Enum	Rótulo na UI	Papel
<code>ADMIN</code>	Administrador	Acesso global. Ignora toda alçada
<code>GESTOR</code>	Gestor	Carteira e comercial. Sem administração de estrutura
<code>OPERACIONAL</code>	Operador	Execução operacional, onboarding, compliance
<code>COMERCIAL</code>	Comercial	Leads e pipeline de vendas, só os próprios cards

Nota de estado real: temos **quatro** valores de enum para **três** tipos de usuário no discurso da empresa.

`COMERCIAL` é anterior a `GESTOR` e foi preservado porque há usuários reais em Production com ele e o seed do funil de Vendas o referencia. Migrar `COMERCIAL` → `GESTOR` é uma decisão pendente, registrada em §26.

5.2 O mecanismo de permissões

Duas camadas, ambas em `lib/permissions.ts`:

`ALL_PERMISSIONS` — 42 chaves, cada uma com `key`, `label` e `group`. Os grupos são: Cockpit, Financeiro, Operacional, CRM, Documentos, Certificados, Compliance, Formulários, Admin.

`DEFAULT_PERMISSIONS` — o que cada papel recebe quando o usuário não tem lista explícita salva.

A função que decide é `hasPermission(permissoes, key, role)`:

1. `role === 'ADMIN'` → true, sempre
2. sem lista explícita → consulta `DEFAULT_PERMISSIONS[role]`
3. com lista explícita → a lista manda

O passo 3 é o que permite conceder ou revogar uma chave individual para um usuário pela tela de Usuários, sem mexer no papel dele.

5.3 Matriz das chaves sensíveis

Apurada executando `hasPermission` para cada papel sem lista explícita:

Chave	ADMIN	GESTOR	OPERACIONAL	COMERCIAL
<code>view_crm</code>	✓	✓	—	—
<code>view_documents</code>	✓	✓	—	—
<code>download_documents</code>	✓	—	—	—
<code>manage_documents</code>	✓	—	—	—
<code>view_certificates</code>	✓	—	—	—
<code>manage_certificates</code>	✓	—	—	—
<code>reveal_certificate_password</code>	✓	—	—	—
<code>view_compliance</code>	✓	✓	✓	—
<code>manage_compliance</code>	✓	—	✓	—
<code>view_forms / manage_forms</code>	✓	—	—	—
<code>manage_automations</code>	✓	—	—	—
<code>admin_funis</code>	✓	—	—	—
<code>view_pipeline / manage_pipeline</code>	✓	✓	—	✓

Três observações que importam:

- `download_documents` é separada de `view_documents`. Listar o que existe não é o mesmo que obter o arquivo.
- `reveal_certificate_password` é chave própria, e só o ADMIN a tem por padrão. Não está embutida em `manage_certificates`.
- **Notificações não têm chave.** Cada usuário vê as suas, filtradas por `destinatarioId`. Uma chave aqui só criaria a chance de configurar errado.

5.4 Alçada por funil

Alçada é o refinamento **dentro** do módulo Pipeline. Cada funil pode ter regras em `PipelinePermissao`, e cada regra vale para **uma role OU um usuário** — nunca os dois.

Seis ações independentes: `ver`, `editar`, `mover`, `criar`, `transferir`, `administrar`. Mais o flag `apenasProprios`.

A resolução está em `resolverAcesso()` (`lib/pipeline.ts`):

1. ADMIN → tudo
2. Há regra para mim? → ela manda (soma das regras que casam)
3. Funil TEM regras, nenhuma minha → sem acesso
4. Funil SEM nenhuma regra → herda o módulo:
 - view_pipeline → ver
 - manage_pipeline → operar
 - administrar → nunca por herança

Detalhe importante e não óbvio: no passo 2, a regra específica vale **por si só**, sem exigir também a chave global `view_pipeline`. Sem isso, a alçada por funil não serviria para nada — o `OPERACIONAL` não tem `view_pipeline` por padrão e nunca alcançaria o funil de Onboarding.

`apenasProprios` só restringe quando **todas** as regras que casam pedem isso: uma concessão nominal mais ampla levanta a restrição herdada da role.

Configuração semeada em Production

Funil	Regra	ver	editar	mover	criar	transferir	administrar	apenasProprios
Vendas	role <code>COMERCIAL</code>	✓	✓	✓	✓	✓	—	✓
Onboarding	role <code>OPERACIONAL</code>	✓	✓	✓	✓	✓	—	—
Operações	role <code>OPERACIONAL</code>	✓	✓	✓	✓	✓	—	—

O `apenasProprios` no Vendas preserva a regra que antes estava embutida no código: o comercial só enxerga os próprios negócios. Agora é configuração.

5.5 Autorização é sempre server-side

O proxy (`proxy.ts`) faz um gate **grosso**: autentica e bloqueia módulos desligados. Ele **não** substitui autorização.

Motivo concreto: `checkAccess` casa rotas por **prefixo**. `/dashboard/pipeline` cobre `/dashboard/pipeline/funis`, então o proxy não consegue separar o quadro da administração de funis. Por isso:

- **toda página administrativa** faz `redirect()` no servidor (8 páginas verificadas: funis, funis/[id], automações, certificados, compliance, documentos, crm, formulários);
- **toda rota de API** verifica `hasPermission` ou `acessoAoFunil` antes de agir.

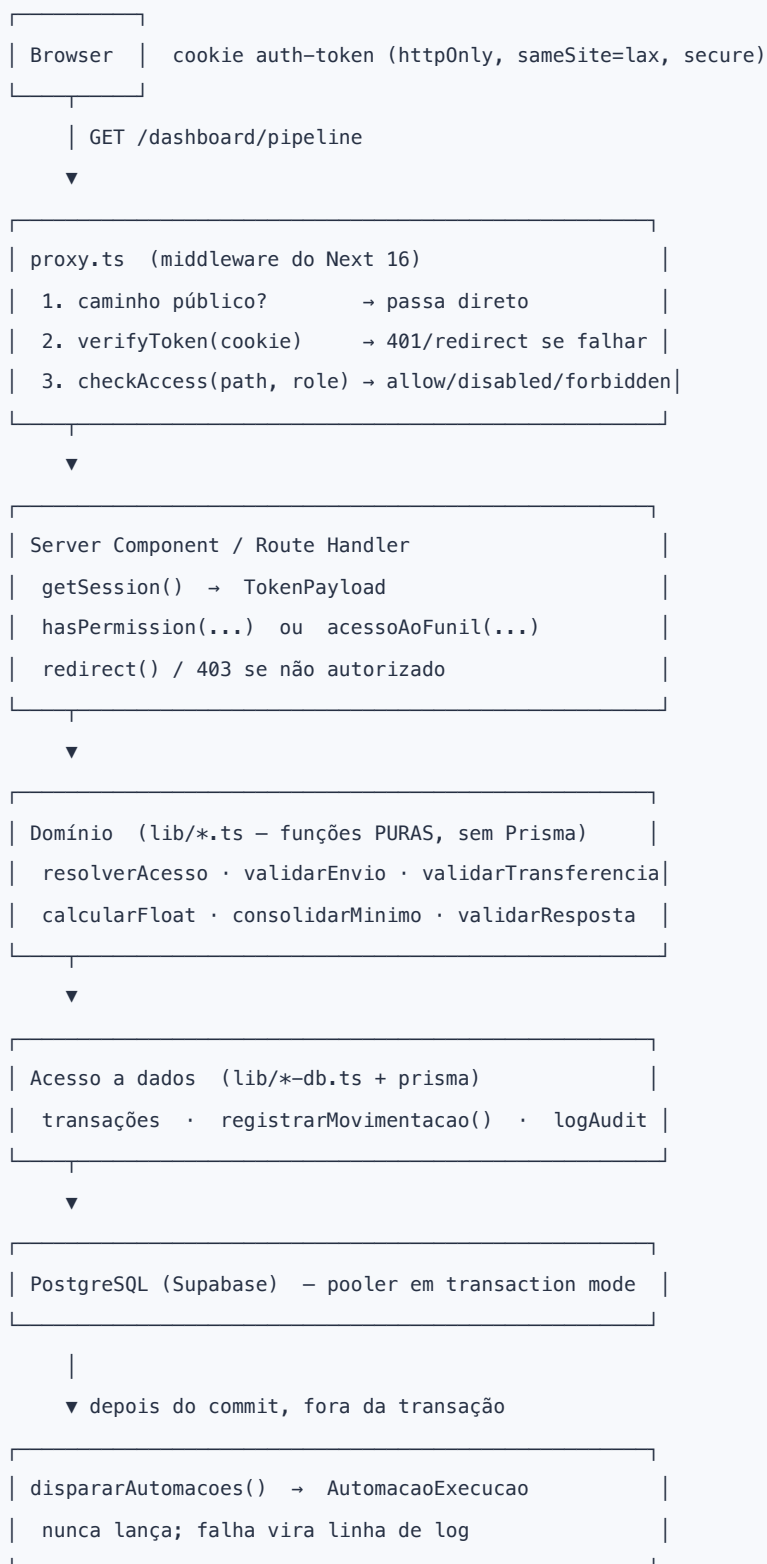
O frontend esconde botões por conveniência, nunca por segurança.

6 Arquitetura técnica

6.1 Stack

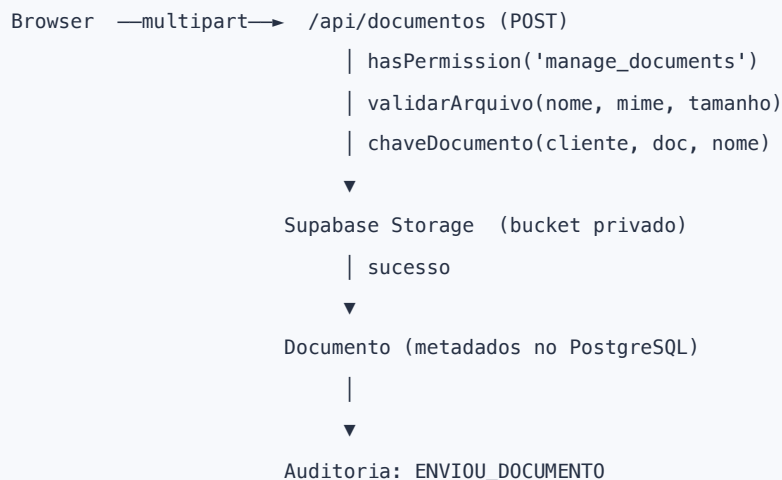
Camada	Tecnologia
Framework	Next.js 16.2.6 (App Router, Turbopack)
UI	React 19.2.4 + TypeScript 5
Estilo	Tailwind CSS 4 com tokens CSS próprios
ORM	Prisma 7.8 com driver adapter <code>@prisma/adapter-pg</code>
Banco	PostgreSQL 17.6 (Supabase gerenciado)
Storage	Supabase Storage , bucket privado
Gráficos	Recharts 3.8
Auth	JWT próprio (<code>jsonwebtoken</code>) + cookie httpOnly
Hash de senha	<code>bcryptjs</code>
Hospedagem	Vercel (Fluid Compute, Node 24)
Testes	<code>node:test</code> via <code>tsx</code> — sem framework externo

6.2 O fluxo de uma requisição autenticada



6.3 O fluxo de um arquivo

UPLOAD



Se a gravação no banco falhar depois do upload, o arquivo é removido do bucket – não fica órfão.

DOWNLOAD



A *service role key* **nunca** sai do servidor. O browser recebe uma URL temporária, não a credencial.

6.4 Separação entre domínio puro e acesso a dados

Uma convenção consistente no `lib/`:

Puro (sem Prisma)	Com Prisma
<code>pipeline.ts</code>	<code>pipeline-db.ts</code>
<code>automacoes.ts</code>	<code>automacoes-db.ts</code>
<code>arquivos.ts</code>	<code>storage.ts</code>
<code>certificados.ts</code> , <code>compliance.ts</code> , <code>formularios.ts</code> , <code>crm.ts</code> , <code>carteira.ts</code> , <code>float.ts</code>	<code>volumetria.ts</code> , <code>kpi.ts</code> , <code>notificacoes.ts</code>

Isso existe por duas razões práticas:

1. **Testabilidade.** As 104 asserções rodam sem banco porque a regra está isolada da consulta.
2. **Segurança de bundle.** `lib/arquivos.ts` foi extraído de `lib/storage.ts` exatamente porque o construtor de formulários precisa dos validadores **no navegador** — e importar `lib/storage` ali arrastaria o SDK do Supabase, e a service role key junto, para o bundle do cliente.

6.5 O registro de módulos

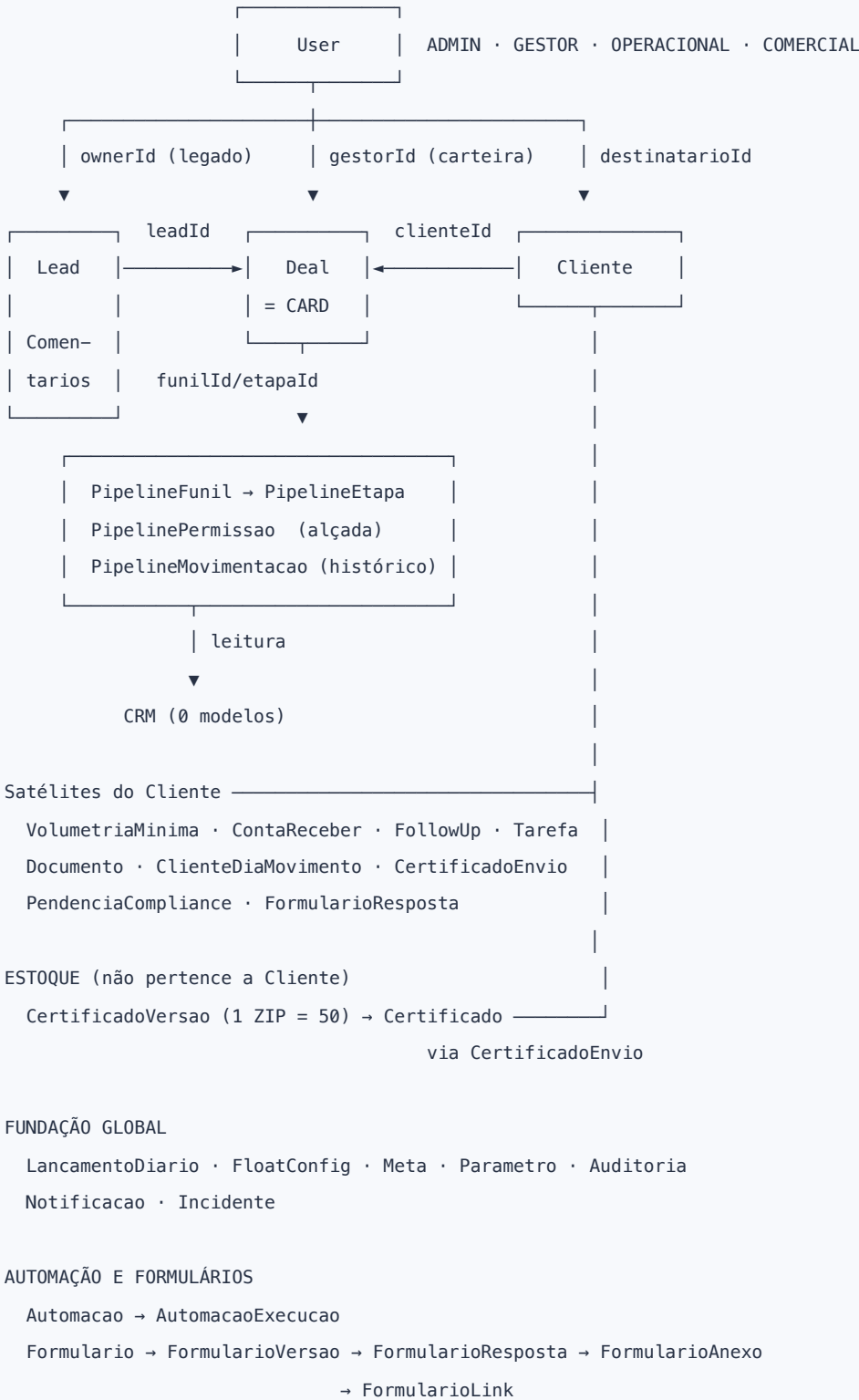
`lib/modules.ts` é a **fonte única de verdade da navegação**. Cada função declara `key`, `label`, `route`, `api[]`, `enabled` e `roles[]`.

Mudar `enabled: false` numa entrada propaga automaticamente para três lugares: o item some da sidebar, a rota é bloqueada no proxy, e os endpoints da função passam a responder 404. Não existe painel de auto-serviço para esses toggles — a mudança é feita em código.

`ALWAYS_ON` lista os caminhos que nunca são bloqueados: autenticação, perfil e notificações.

7 Catálogo do banco de dados

7.1 Visão conceitual do domínio



7.2 Classificação das 35 entidades

Classe	Modelos
Centrais	User , Cliente , Lead , Deal
Transacionais	LancamentoDiario , ContaReceber , Tarefa , Incidente , CertificadoEnvio , PendenciaCompliance , FormularioResposta , ClienteDiaMovimento
Configuração	PipelineFunil , PipelineEtapa , Automacao , Formulario , FormularioVersao , Parametro , FloatConfig , Meta , VolumetriaMinima
Segurança / alçada	PipelinePermissao , Certificado (senha cifrada)
Histórico	PipelineMovimentacao , PendenciaEvento , AutomacaoExecucao , Activity , LeadComentario
Auditoria	Auditoria
Comunicação	Notificacao
Ativos	Documento , CertificadoVersao , FormularioAnexo , FormularioLink
Relacionamento	FollowUp

7.3 Entidades centrais

User — 31 campos

Pessoa que usa o sistema. Campos-chave: `email` (único), `password` (bcrypt), `role`, `active`, `permissoes` (lista explícita opcional, serializada).

Tem 13 relações de saída, incluindo `clientesGeridos` (carteira), `notificacoes`, `pipelinePermissoes`, `movimentacoes` e `auditorias`. Desativar (`active: false`) é o mecanismo de desligamento — usuários não são excluídos, para que o histórico continue resolvendo o nome.

Cliente — 39 campos

O cliente da Bass Pago. Além dos dados cadastrais (`nome`, `cnpj`, `email`, `telefone`), guarda a **configuração comercial**: `modeloOperacional` (API / WHITE_LABEL / BAAS), `segmento`, `operacao`, `scoreRisco`, e a **expectativa**: `tpvEsperado`, `qtdTransacoesEsperada`, `qtdMedEsperada`, `receitaPrevistaMensal`.

Dois vínculos com `User`:

- `ownerId` (NOT NULL) — legado, quem cadastrou;
- `gestorId` (nullable) — o **Gestor responsável pela carteira**, introduzido na v12. Deliberadamente **não** substitui `ownerId`.

`status` aceita `ATIVO`, `INATIVO`, `PROSPECCAO`, `ENCERRADO`, `STANDBY`.

Lead — 21 campos

Contato comercial antes de virar cliente. O cadastro revisado do CRM usa: `company`, `name` (executivo), `cnpj`, `phone`, `email`, `segmento`, `canal`, `notes`, `ownerId` (responsável). Tem `comentarios` (`LeadComentario`) com autor e data/hora — separados de `notes`, que é a observação livre do cadastro.

Deal — 24 campos — o card do Pipeline

É a peça mais reaproveitada do sistema. Carrega `title`, `value`, `probability`, `stage` (legado), `ownerId`, `leadId`, e desde a v11: `funilId`, `etapaId`, `clienteId`.

Um único `Deal` atravessa **Vendas** → **Onboarding** → **Operações**. Não se cria um card por funil; é isso que mantém lead, cliente, dono e atividades associados e permite uma linha do tempo contínua.

`stage` permanece como campo **legado**, sincronizado apenas enquanto o card estiver no funil de Vendas, cujas etapas nasceram dos próprios valores do enum. `app/api/relatorios` usa esse campo e foi escopado ao funil de Vendas.

7.4 Fundação de indicadores

LancamentoDiario — a fonte oficial dos 5 indicadores

Grão: **uma data**. Campos: `receitaTarifaria`, `tpv`, `saldoEmConta`, `qtdTransacoes`, `qtdMed`. `data` é `@unique @db.Date`.

O comentário no schema é explícito: *"Nenhum outro modelo grava TPV, receita tarifária, saldo, transações ou MEDs."* É a regra de fonte única do §1.1 materializada.

FloatConfig — o multiplicador versionado

`multiplicador` + `vigenciaInicio` (única). Versionado por vigência para que alterar o valor hoje **não reescreva o Float de períodos já fechados**.

Meta e Parametro

`Meta` guarda apenas o **alvo** (`tipo`, `valor`, `periodo`), único por (`tipo`, `periodo`). O realizado vem sempre do `LancamentoDiario`.

`Parametro` são limiares configuráveis por tela, agrupados. Em Production há 10: `META_RECEITA_ALTO/CRITICO`, `VOLUME_MINIMO_ALTO/CRITICO`, `MED_PORCENT_ALTO/CRITICO`, `SATISFACAO_ALTO/CRITICO`, `DOWNTIME_CRITICO_MIN`, `SCORE_RISCO_ALERTA`.

7.5 Pipeline — 4 modelos

Modelo	Papel
PipelineFunil	nome único, <code>area</code> , <code>ordem</code> , <code>ativo</code> , <code>exigeCliente</code>
PipelineEtapa	<code>funilId</code> , <code>nome</code> , <code>ordem</code> , <code>cor</code> , <code>ativo</code> , <code>tipo</code> (NORMAL/GANHOS/PERDIDO)
PipelinePermissao	alçada; 6 booleanas + <code>apenasProprios</code> ; role ou <code>userId</code>
PipelineMovimentacao	histórico estruturado com origem, destino, usuário, timestamp

Dois detalhes de modelagem que valem registro:

- `ordem` fica fora de qualquer índice único. Um unique impediria trocar duas etapas de posição sem um valor intermediário.
- `PipelinePermissao` tem dois uniques: `(funilId, role)` e `(funilId, userId)`. Convivem porque NULLs são distintos no Postgres — linhas de role têm `userId` nulo e vice-versa.

7.6 Certificados — 3 modelos, dois conceitos

Modelo	O que é	Quantidade
CertificadoVersao	o ZIP / estoque global — não tem <code>clienteId</code>	sempre 50
Certificado	a unidade, com senha cifrada	50 por versão
CertificadoEnvio	a distribuição ao cliente	1 ou 10

A coluna que separa os dois conceitos é `Certificado.envioId`: nula significa *ainda em estoque*. É por isso que "quantos sobraram nesta versão?" não precisa de nenhum contador denormalizado.

Unicidade: `(versaoId, numero)`. A numeração é **relativa à versão** — 1 a 50 em cada ZIP, nunca uma sequência global.

7.7 Demais modelos

Modelo	Finalidade
Notificacao	mensagem dirigida a um usuário; contexto polimórfico leve (entidade + entidadeId + href), sem FK porque aponta para sete modelos
Documento	metadados do arquivo; storageKey único; ativo para inativação lógica
ClienteDiaMovimento	booleano por cliente-dia; @@unique([clienteId, data])
PendenciaCompliance	cliente, motivo, criticidade (reusa IncidenteCriticidade), prazo, responsável, status
PendenciaEvento	timeline da pendência: statusDe → statusPara , comentário, autor
Automacao	regra no-code; escopo nulo = qualquer funil/etapa; condicao em JSONB
AutomacaoExecucao	log de disparo: status, contexto, resultado ou erro
Formulario / FormularioVersao	formulário e seus snapshots imutáveis
FormularioLink	token aleatório, expiração, limite de usos, revogação
FormularioResposta	valores em JSONB, aceite, assinatura, IP
FormularioAnexo	ponte 1:1 entre resposta e Documento
Auditoria	trilha global: ação, entidade, entidadeId, detalhes, usuário
ContaReceber	cobranças do cliente; alimenta as linhas Setup e Serviços
FollowUp	rotina de relacionamento: recorrência, dia da semana, janela
Incidente	incidentes operacionais com downtime e criticidade
Activity	atividades ligadas a lead/deal (modelo original, pouco usado hoje)

7.8 Enums que carregam regra de negócio

Enum	Por que importa
<code>EtapaTipo</code>	<code>GANHO</code> / <code>PERDIDO</code> encerram o card (<code>closedAt</code>) e saem das colunas ativas
<code>MovimentacaoTipo</code>	distingue criação, movimento e transferência — é o que o CRM usa para separar métricas
<code>CertificadoEnvioTipo</code>	<code>UNICO</code> = 1, <code>LOTE</code> = 10. A quantidade é derivada do tipo, não digitada
<code>PendenciaStatus</code>	governa a máquina de estados de compliance (§17.2)
<code>AutomacaoGatilho</code> / <code>AutomacaoAcao</code>	o vocabulário fechado do motor no-code
<code>Segmento</code>	4 valores atuais (Crypto/Exchanges, Remessa/FX, Gateway, BaaS) + Telecom, ERP, iGaming, SaaS; os 4 últimos são legado preservado
<code>Canal</code>	Outbound, Indicação, Inbound, Eventos, Outro
<code>MetaTipo</code>	5 tipos atuais + 3 legados mantidos só para leitura

8 Dashboard (Cockpit)

A tela inicial. Responde "como estamos indo neste mês" em uma olhada.

8.1 Os KPIs e suas origens

Todos os cards vêm de `kpisDoPeriodo(periodo)` em `lib/kpi.ts`, que lê `LancamentoDiario` do mês. Cada card traz valor, variação contra o mês anterior e um sparkline de 12 meses.

Indicador	Fórmula / origem	O que decide
TPV	soma de <code>LancamentoDiario.tpv</code>	escala da operação
Receita Tarifária	soma de <code>receitaTarifaria</code>	o que a tarifa gerou
Float	<code>calcularFloat(saldos, vigências)</code> — §9	quanto o caixa rende parado
Faturamento	soma das 5 linhas (<code>linhasReceita</code>)	receita total do período
Take Rate	<code>receitaTarifaria ÷ tpv</code>	eficiência de monetização
Saldo Médio	média de <code>saldoEmConta</code> nos dias lançados	base do Float
% de MEDs	<code>qtdMed ÷ qtdTransacoes</code>	qualidade da operação
Transações	soma de <code>qtdTransacoes</code>	volume operacional

8.2 As 5 linhas de faturamento

`linhasReceita()` compõe o faturamento a partir de fontes distintas:

Linha	Origem
<code>tarifario</code>	<code>LancamentoDiario.receitaTarifaria</code>
<code>float</code>	derivado (§9)
<code>sustentacao</code>	<code>Cliente.mensalidadeApi</code> + <code>Cliente.sustentacaoWhiteLabel</code>
<code>setup</code>	<code>ContaReceber</code> do tipo setup
<code>servicos</code>	<code>ContaReceber</code> do tipo serviços

8.3 Metas e Volumetria no Dashboard

Metas: `metasDoPeriodo()` cruza o alvo (`Meta.valor`) com o realizado vindo do `LancamentoDiario`, e calcula atingimento. Quando não há realizado, o atingimento é `null` — não zero.

Volumetria: um painel com o mínimo consolidado do mês, o realizado e o status, indicando se o mínimo veio da soma de N clientes ou de contrato geral legado (§10).

8.4 Painel de Insights

`components/dashboard/InsightsPanel.tsx` roda 8 regras de `lib/insights.ts` e mostra as que têm algo a dizer:

`ruleReceitaMoM` · `ruleFloatTrend` · `ruleInadimplencia` · `ruleNovosClientes` · `ruleChurn` ·
`ruleTakeRateMoM` · `ruleVolumetria` · `ruleDiasSemLancamento`

Cada regra devolve `null` quando não há sinal. O painel não inventa comentário.

8.5 Alertas — diferentes de Notificações

`/dashboard/alertas` roda verificações automáticas e derivadas sobre o período: volumetria não atingida, dias sem lançamento, multiplicador de Float não configurado, cobranças vencidas, incidentes em aberto.

Alerta é uma verificação sem dono e sem estado de leitura. Notificação tem destinatário e lida/não lida. São mecanismos distintos (§14).

9 Float — o indicador mais sutil do sistema

9.1 O conceito

Float é o rendimento do dinheiro que **fica parado em conta durante a noite**. Em instituição de pagamentos, esse saldo é uma fonte de receita real.

$$\text{FLOAT} = \text{SALDO QUE DORME} \times \text{MULTIPLICADOR}$$

9.2 O que é "saldo que dorme"

Entre dois dias consecutivos D e D+1, o valor que efetivamente atravessou a noite é o **MENOR** dos dois saldos:

$$\text{dorme}(D \rightarrow D+1) = \min(\text{saldo}(D), \text{saldo}(D+1))$$

A razão está no código:

- o que saiu antes da virada não dormiu;
- o que entrou só no dia seguinte não estava lá durante a noite.

9.3 Exemplo numérico

Dia	Saldo em conta	Dormiu (para o dia seguinte)
01	10.000.000	$\min(10M, 12M) = 10.000.000$
02	12.000.000	$\min(12M, 9M) = 9.000.000$
03	9.000.000	— (último dia do recorte)

Com multiplicador de 0,05% ao dia:

$$\begin{aligned}\text{noite 01} \rightarrow \text{02} &: 10.000.000 \times 0,0005 = 5.000 \\ \text{noite 02} \rightarrow \text{03} &: 9.000.000 \times 0,0005 = 4.500 \\ \text{FLOAT do período} &= 9.500\end{aligned}$$

Os 2 milhões extras do dia 02 **não rendem na primeira noite**, porque chegaram depois da virada.

9.4 Gaps

Dias sem lançamento **não geram float**. Sem saldo registrado não há como saber o que havia em conta, e inventar um valor seria exatamente o fallback silencioso que a refatoração dos KPIs eliminou.

`noitesDoPeriodo()` só forma uma noite entre **dias de calendário consecutivos**. Um buraco no meio do mês simplesmente não produz noite.

9.5 Multiplicador versionado

`multiplicadorEm(data, vigencias)` escolhe a vigência mais recente que já começou. Se a data for anterior a qualquer configuração, devolve `null` e a noite não rende — em vez de assumir um valor arbitrário.

Isso garante que **alterar o multiplicador hoje não reescreve o Float de períodos fechados**.

9.6 Por que é derivado e nunca lançado

Se alguém pudesse digitar o Float, ele deixaria de ser medição e viraria opinião — e o faturamento passaria a conter um número que ninguém consegue auditar. Sendo derivado, ele é sempre reproduzível a partir do saldo diário e da configuração vigente.

Um aviso relacionado: se `FloatConfig` estiver vazio, um alerta crítico aparece na tela de Alertas, porque **sem multiplicador o faturamento fica subestimado**.

10 Volumetria mínima por cliente

10.1 O conceito

Volumetria mínima é uma **cláusula contratual**: o cliente X se compromete a gerar no mínimo N transações por mês, de MM/AAAA até MM/AAAA (ou por prazo indeterminado).

Modelo: `VolumetriaMinima`, com `clienteId`, `periodo` (início da vigência), `vigenciaFim` (nulo = indeterminado), `qtdMinima`, `ativo` e `notas`.

10.2 Volumetria ≠ TPV

	Volumetria	TPV
Mede	quantidade de transações	valor financeiro
Natureza	exigência contratual	resultado realizado
Granularidade	por cliente	agregado da empresa
Origem	negociação comercial	<code>LancamentoDiario</code>

10.3 Contrato individual × consolidado

Esta é a parte que exige atenção.

O realizado (`qtdTransacoes`) vem do `LancamentoDiario`, que é **global por data**. Não existe transação por cliente. Logo, **não há atingimento por cliente** — só o consolidado do mês:

```
mínimo do mês = Σ qtdMinima dos contratos VIGENTES naquele mês
realizado      = Σ qtdTransacoes do mês (LancamentoDiario)
status         = ATINGIDO | NAO_ATINGIDO | EM_ACOMPANHAMENTO | SEM_DADOS
```

`EM_ACOMPANHAMENTO` existe para o mês corrente: ainda não fechou, então "não atingido" seria uma conclusão precipitada.

Se nenhum contrato de cliente cobre o período — caso dos meses anteriores à funcionalidade — cai no **contrato geral legado** (`clienteId` nulo), para que os alertas históricos não mudem de resposta.

10.4 Status de um contrato

`statusContrato()` deriva quatro estados, sem coluna de status:

Estado	Regra
INATIVA	<code>ativo = false</code>
PROGRAMADA	<code>início > mês de referência</code>
ENCERRADA	<code>fim < mês de referência</code>
VIGENTE	os demais casos

Períodos são strings `"YYYY-MM"` — a ordem lexicográfica é a cronológica, o que torna as comparações de vigência triviais.

10.5 Regra de não sobreposição

Duas vigências **ativas** do mesmo cliente não podem se sobrepor. Motivo concreto: duas exigências válidas no mesmo mês fariam a soma consolidada contar o mesmo contrato duas vezes.

Validado na criação, na edição **e na reativação** — reativar pode recriar uma sobreposição que a inativação havia resolvido.

10.6 O problema empresarial resolvido

Contratos com volumetria mínima geram direito a cobrança quando o cliente não entrega o volume. Antes, isso vivia em contrato e planilha, e ninguém sabia em tempo real se o conjunto da carteira estava dentro do contratado. Agora o Cockpit responde isso no dia 10 do mês, não no fechamento.

11 Carteira Comercial

11.1 O que a tela mostra

`/dashboard/carteira` lista os clientes com: nome e CNPJ, segmento, modelo operacional, score de risco, status, TPV esperado, receita prevista, **Gestor** e a **faixa dos últimos 5 dias**.

Filtros: busca por nome, status, modelo operacional, segmento.

11.2 Gestor e o vínculo legado

Dois campos apontam para `User` :

- `gestorId` — o Gestor responsável pela carteira (introduzido na v12, nullable). É o que a coluna "Gestor" exibe.
- `ownerId` — vínculo legado (NOT NULL), preservado. A tela cai para ele quando não há gestor definido.

O papel de *Farmer* corresponde funcionalmente ao Gestor: quem cultiva a conta depois de fechada. **Não existe campo separado chamado "Farmer"** — é o mesmo `gestorId` .

11.3 O indicador dos últimos 5 dias

✗ ✓ ✓ ✗ ✓

Cada símbolo é um dia. O modelo é `ClienteDiaMovimento` , com exatamente um dado útil: `movimentou: Boolean` .

Três estados com um booleano. A ausência de linha significa *sem registro*:

Símbolo	Estado	Significado
✓	SIM	houve movimentação
✗	NAO	não houve movimentação
.	SEM_REGISTRO	ninguém preencheu aquele dia

Isso é deliberado: **um dia não preenchido não pode parecer um dia sem movimento**. Marcar é clicar no símbolo; o `upsert` é imediato e auditado.

Os dias são calculados em **UTC** (`ultimosDias()` em `lib/carteira.ts`), para que "últimos 5 dias" não mude de resultado conforme o fuso de quem abre a tela.

11.4 Por que esse indicador existe — e o que ele NÃO é

Um gestor precisa saber se um cliente **parou de operar** antes que isso vire churn. O sinal mais precoce não é o valor caindo — é o cliente simplesmente deixando de transacionar.

Não é TPV por cliente. O modelo não tem nenhum campo de valor, quantidade ou moeda. É um sinal binário de atividade, e essa restrição é arquitetural, não acidental.

11.5 Expectativa e potencial

`Cliente` guarda a expectativa negociada: `tpvEsperado` , `qtdTransacoesEsperada` , `qtdMedEsperada` , `receitaPrevistaMensal` , além de `descontoPercent` e `overpricePercent` . São **valores de referência comercial**, não medições.

12 CRM

12.1 O cadastro

O cadastro de Lead usa exatamente estes campos: **Empresa** (`company`), **Nome do executivo** (`name`), **CNPJ**, **Celular** (`phone`), **E-mail**, **Segmento**, **Canal**, **Responsável** (`ownerId`), **Observações** (`notes`).

Segmentos: Crypto/Exchanges/PSAV/P2P/OTC · Remessa/FX/Crossborder/Pagamentos Internacionais · Gateway de Pagamentos · Telecom · ERP · iGaming · SaaS · BaaS.

Canais: Outbound · Indicação · Inbound · Eventos · Outro.

Comentários ficam em `LeadComentario`, com autor e data/hora — separados de `notes`, que é a observação livre do cadastro.

12.2 Por que o CRM não tem entidade própria

Porque tudo que ele mostra **já existe**. `lib/crm.ts` é composto apenas de funções puras que recebem linhas já carregadas e devolvem números. Nenhum `INSERT`, nenhuma tabela de agregados.

Uma tabela de métricas precisaria ser recalculada, versionada e sincronizada — e divergiria da verdade no primeiro bug.

12.3 As métricas, uma a uma

Tempo médio por etapa

```
tempoMedioPorEtapa(movimentos, agora)
```

A saída de uma etapa é a **próxima movimentação do mesmo card**. O último trecho fica em aberto — o card ainda está lá — e conta **até agora**.

Sem essa última parte, a etapa onde tudo empaca apareceria como a mais rápida do funil, porque cards parados nunca produziram um intervalo fechado.

Conversão por etapa

```
conversaoPorEtapa(movimentos, ordemEtapas)
```

Dos cards que **entraram** numa etapa, quantos chegaram a alguma etapa **posterior** do mesmo funil. Etapa sem entrada tem taxa `null`, nunca zero.

Conversão por responsável

```
conversaoPorResponsavel(cards, etapasGanho, etapasPerda)
```

A taxa considera **apenas os cards já decididos** (ganhos + perdas). Incluir os abertos puniria quem tem pipeline cheio.

Conversão entre funis

`conversaoEntreFunis(movimentos)` — agrega as transferências por par origem→destino. É o que mostra quantos negócios de Vendas efetivamente chegaram ao Onboarding.

Ciclo médio

`cicloMedioDias(cards)` — média de `closedAt - createdAt` dos cards encerrados. `null` quando nenhum fechou.

Gargalos

`gargalos(tempo, volumePorEtapa)`

Etapa cujo tempo médio passa do **dobro da mediana do funil** e que **ainda tem card parado**. O dobro da mediana evita apontar como gargalo uma etapa naturalmente mais longa num funil curto. Com menos de 3 etapas medidas, não opina.

Volume por etapa

Contagem de cards por etapa, exibida com barra proporcional.

12.4 Escopo de leitura

A tela só oferece os funis que a alçada do usuário permite ver (`funisVisiveis`). A analítica **não é porta lateral** para dados de um funil fechado.

13 Pipeline multi-funil

13.1 Os três funis semeados

Funil	Área	Etapas
Vendas	Comercial	Prospecção · Qualificação · Proposta · Negociação · Fechamento · Ganho · Perdido
Onboarding	Operacional	Kickoff · Contrato · Ajuste de Taxas · Integração · Homologação · Go Live
Operações	Operacional	Ativação · Monitoramento · Sustentação

Ganho e Perdido têm `tipo` `GANHO` / `PERDIDO` : encerram o card (`closedAt`) e saem das colunas ativas do quadro.

Onboarding e Operações têm `exigeCliente = true` .

13.2 Vocabulário

Termo	No sistema
Funil	<code>PipelineFunil</code> — um processo com etapas e alçadas próprias
Etapas	<code>PipelineEtapa</code> — uma coluna do quadro
Card	<code>Deal</code> — a unidade que se move
Responsável	<code>Deal.ownerId</code>
Movimentação	<code>PipelineMovimentacao</code> — cada mudança registrada
Transferência	mudança de funil, não só de etapa

13.3 Mover x Transferir

Mover (`POST /api/pipeline/cards/[id]/mover`) troca de etapa **dentro do mesmo funil**. Exige `mover` . Valida etapa ativa e do mesmo funil.

Transferir (`POST .../transferir`) muda de funil. Exige `transferir` no funil de origem e `criar` no de destino — transferir é também criar no destino.

13.4 Uma transferência, passo a passo

Cenário: card em **Vendas / Fechamento** vai para **Onboarding / Kickoff**.

1. Usuário clica em [Transferir] no card
2. Modal:
 - funil destino → só os funis com permissão `criar`
 - etapa inicial → só etapas ativas do destino
 - cliente → pedido, porque Onboarding tem exigeCliente
 - observação → opcional
3. Confirmar

O servidor então:

```
validarTransferencia()
  | destino ativo?
  | etapa pertence ao destino?
  | etapa ativa?
  | origem ≠ destino?
  | cliente presente, se o funil exigir?

TRANSAÇÃO
  | Deal.update → funilId, etapaId, clienteId, stage(se Vendas), closedAt
  | PipelineMovimentacao.create
    tipo          = TRANSFERENCIA_FUNIL
    funilOrigemId = Vendas
    etapaOrigemId = Fechamento
    funilDestinoId = Onboarding
    etapaDestinoId = Kickoff
    userId        = quem transferiu
    observacao    = texto livre

COMMIT

DEPOIS DO COMMIT
  | Auditoria: TRANSFERIU_CARD
  | Notificacao para quem tem `ver` no Onboarding (menos quem transferiu)
  | dispararAutomacoes(CARD_TRANSFERIDO)
```

O **Deal** não é recriado. É o mesmo registro mudando de funil — por isso lead, cliente, valor, dono e atividades seguem associados por construção.

13.5 Administração de funis

`/dashboard/pipeline/funis` (lista, criar, ordenar, inativar) e `/dashboard/pipeline/funis/[id]` (editar, etapas, permissões).

Regras que o código impõe:

- **Não existe exclusão de funil nem de etapa** — só inativação. Excluir deixaria o histórico apontando para o vazio.

- **Inativar etapa com cards exige escolher a etapa de destino** deles. Os cards são realocados e cada movimento vira uma `PipelineMovimentacao` com a observação de que foi realocação.
- **Reordenar** grava a nova sequência numa transação; `validarReordenacao` exige exatamente o conjunto atual — sem faltar, sobrar ou repetir.

13.6 Permissões, uma vez mais

O quadro oferece apenas os funis com `ver`. Arrastar exige `mover`; "+ Adicionar" exige `criar`; o botão Transferir exige `transferir`. Com `apenasProprios`, o `where` da consulta ganha `ownerId: session.userId`.

14 Notificações

14.1 O que é — e o que não é

Notificação é mensagem dirigida a **uma pessoa**, com estado de leitura e um contexto para onde voltar.

Alerta (§8.5) é verificação automática sobre o período, sem dono e sem leitura.

Confundir os dois foi um risco explícito no desenho; por isso são módulos separados.

14.2 O modelo

`Notificacao` : `destinatarioId` , `titulo` , `mensagem` , `origem` , `entidade` , `entidadeId` , `href` , `lidaEm` , `createdAt` .

O contexto é **polimórfico leve** — sem FK, porque aponta para sete modelos diferentes. `href` é o link que a central abre.

Origens: `PIPELINE` , `AUTOMACAO` , `COMPLIANCE` , `FORMULARIO` , `CERTIFICADO` , `SISTEMA` .

14.3 De onde nascem, hoje

Três produtores reais no código:

1. **Transferência de card** — quem tem `ver` no funil de destino é avisado, menos quem transferiu (ninguém é notificado do próprio ato).
2. **Compliance** — ao abrir pendência ou reatribuir responsável.
3. **Automações** — ação `NOTIFICAR` , e também como efeito de `CRIAR_TAREFA` e `ABRIR_PENDENCIA` .
4. **Envio de certificados** — gestor e owner do cliente são avisados.

14.4 Interface

- **Sino no Topbar** (`SinoNotificacoes`): contador de não lidas, lista das 8 mais recentes, "marcar todas como lidas", fecha com Esc ou clique fora. Faz *polling* a cada 60 segundos.
- **Central** (`/dashboard/notificacoes`): histórico completo, filtro de não lidas, marcar/desmarcar individual, botão para abrir o contexto.

14.5 Segurança

`notificar()` **nunca derruba a ação que a originou** — uma transferência válida não pode falhar porque o aviso não saiu. Erros são engolidos e a função devolve quantas foram criadas.

A leitura usa `updateMany` com `destinatarioId` no `where` : um id de outro usuário simplesmente não casa, em vez de vaziar a existência da notificação por um 403.

15 Documentos

15.1 Categorias e formatos

Categorias (`DocumentoCategoria`): Contrato · KYC/KYB · Certificado · Comprovante · Comercial · Financeiro · Outros.

Formatos aceitos (12): `pdf` , `doc` , `docx` , `xls` , `xlsx` , `csv` , `txt` , `jpg` , `jpeg` , `png` , `webp` , `zip` . Limite: **25 MB**.

15.2 Upload

A tela (`/dashboard/documentos`) tem **drag & drop**, seleção por arquivo, **barra de progresso real** e **cancelamento**. O progresso usa `XMLHttpRequest` em vez de `fetch` — é o que dá evento de upload e `abort()` .

O arquivo passa **inteiro pelo servidor**, de propósito: é o único jeito de validar extensão, MIME, tamanho e autorização antes de qualquer byte chegar ao bucket, e mantém a service role key fora do navegador.

15.3 Validação

`validarArquivo(nome, mime, tamanho)` em `lib/arquivos.ts` verifica os três:

- extensão conhecida;
- **MIME compatível com a extensão** — extensão sozinha se falsifica;
- tamanho entre 1 byte e 25 MB.

`nomeSeguro()` normaliza acentos, troca tudo que não é `[a-zA-Z0-9._-]` por `_` , **corta pontos iniciais** e limita a 120 caracteres. A chave final é `clientes/{clienteId}/{documentoId}/{nome}` — sempre sob o prefixo do cliente, sem possibilidade de travessia.

15.4 Download

```
GET /api/documentos/[id]/download
  | hasPermission('download_documents') → 403 se não
  | documento existe? está ativo?      → 404 / 409
  | urlAssinada(storageKey, 60s)
  | Auditoria: BAIXOU_DOCUMENTO
  | { url, nome, expiraEm: 60 }
```

60 segundos é **tempo de clicar, não de compartilhar**.

15.5 Inativação, não exclusão

`PATCH /api/documentos/[id]` alterna `ativo`. **Não existe exclusão física pela interface** — perder o registro apagaria o rastro de que o arquivo existiu, que é justamente o que a auditoria precisa preservar.

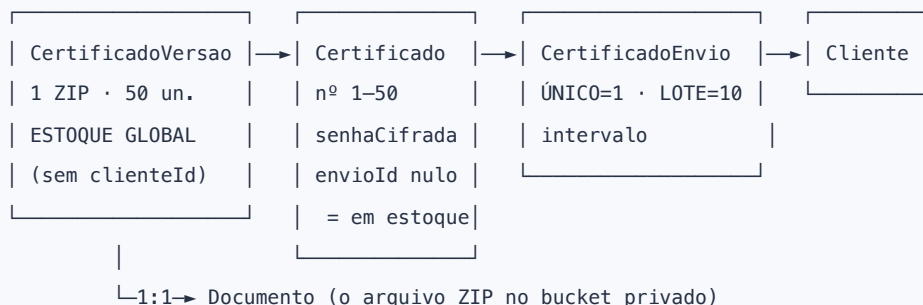
A única remoção física acontece quando o upload sobe mas a gravação no banco falha: aí o arquivo é apagado para não ficar órfão.

15.6 Por que os bytes não ficam no PostgreSQL

Bytes em banco relacional inflam backup, complicam replicação e transformam cada download numa consulta pesada. O PostgreSQL guarda a chave; o Storage guarda o arquivo.

16 Gestão de Certificados

16.1 Dois conceitos que não se misturam



- **Versão / ZIP = estoque.** Sempre 50 certificados. **Não tem** `clienteId` — é estoque global da empresa.
- **Envio = distribuição.** Único (1 certificado) ou Lote (10).
- O vínculo com o cliente acontece **apenas no envio**.

16.2 Numeração relativa à versão

Cada ZIP numera de **1 a 50**. Não há sequência global. A unicidade é `(versaoId, numero)`.

Consequência prática que importa: um intervalo só faz sentido citado junto da versão. "certificados 1–10" é ambíguo, porque 1–10 existe em toda versão. Por isso `rotuloIntervalo()` sempre produz:

```
Versão 002 – certificados 1–10
Versão 002 – certificado 11
```

16.3 Criação de uma versão

`POST /api/certificados/versoes` cria a versão e **os 50 certificados de uma vez**, cada um com senha própria — gerada (`gerarSenha()`) ou informada, na ordem 1 a 50. A senha em claro **nunca** é persistida nem devolvida na resposta.

`gerarSenha()` usa um alfabeto sem caracteres que se confundem lidos em voz alta ou copiados à mão: sem `0`, `0`, `1`, `l`, `I`.

16.4 Registro de um envio

`validarEnvio()` verifica, nesta ordem:

1. números entre 1 e 50 — "a numeração é relativa à versão";
2. quantidade **exata** do tipo: 1 para único, 10 para lote;
3. todos os números pedidos ainda em estoque.

Na transação: cria o `CertificadoEnvio`, marca os certificados escolhidos com `envioId` e `status = ENVIADO`, e — se o estoque zerar — muda a versão para `ESGOTADA` automaticamente.

Depois: auditoria `ENVIOU_CERTIFICADOS` e notificação para gestor e owner do cliente.

16.5 Cancelamento

`PATCH /api/certificados/envios/[id]` com `status: CANCELADO` devolve os certificados ao estoque (`envioId = null`, `status = DISPONIVEL`) e reabre a versão se estava esgotada.

Nada é apagado. O envio permanece com status `CANCELADO`, porque o histórico de que ele existiu é o que a auditoria precisa.

16.6 Criptografia — AES-256-GCM

`lib/crypto-certificado.ts`.

Por que cifragem reversível, e não hash

Senha de certificado precisa ser **lida de volta** para ser entregue ao cliente. Hash é irreversível por construção — não serve.

Por que GCM

Além de cifrar, **autentica**. Um registro adulterado no banco falha ao decifrar em vez de devolver lixo silenciosamente. Os testes cobrem exatamente esse caso.

Formato armazenado

`iv : tag : conteúdo` — os três em base64url, separados por ":"

<code>iv</code>	12 bytes	aleatório por operação
<code>tag</code>	16 bytes	authentication tag do GCM
<code>chave</code>	32 bytes	de <code>CERTIFICADO_ENCRYPTION_KEY</code>

IV aleatório por operação significa que cifrar duas vezes a mesma senha produz resultados diferentes — senhas iguais não são identificáveis no banco. Há teste para isso.

A chave aceita hex de 64 caracteres (32 bytes, o ideal) ou qualquer string, derivada por SHA-256. Em produção, `openssl rand -hex 32`.

Revelação da senha

`POST /api/certificados/[id]/senha`

POST, não GET — de propósito. Revelar é ação com efeito (deixa rastro), e não deve ser disparável por prefetch, histórico ou link colado.

A sequência:

1. `hasPermission('reveal_certificate_password')`
 - ↳ negado → grava `NEGOU_REVELACAO_SENHA` e responde 403
(quem tentou ver o que não pode é informação de segurança)
2. decifra
3. GRAVA A AUDITORIA – antes de responder
 - ↳ se a gravação falhar, a senha não sai
4. devolve { senha, referencia } no corpo

Na interface, a senha vai **direto para a área de transferência** e nunca é escrita na tela. Só a referência do certificado aparece como confirmação.

Onde a senha nunca aparece

`senhaCifrada` fica **fora do `select`** em toda listagem — a rota de detalhe da versão devolve `id`, `numero`, `status` e `envioId`, e nada mais. Nunca em log, nunca em URL.

16.7 O risco operacional que isso resolve

Certificados digitais e suas senhas costumam circular por e-mail e planilha compartilhada. O resultado é previsível: ninguém sabe qual certificado foi para qual cliente, quantos sobraram, nem quem viu a senha.

Esta arquitetura responde as quatro perguntas: **o que existe** (estoque por versão), **para quem foi** (envio), **quem viu a senha** (auditoria) e **quanto resta** (`envioId` nulo).

17 Compliance

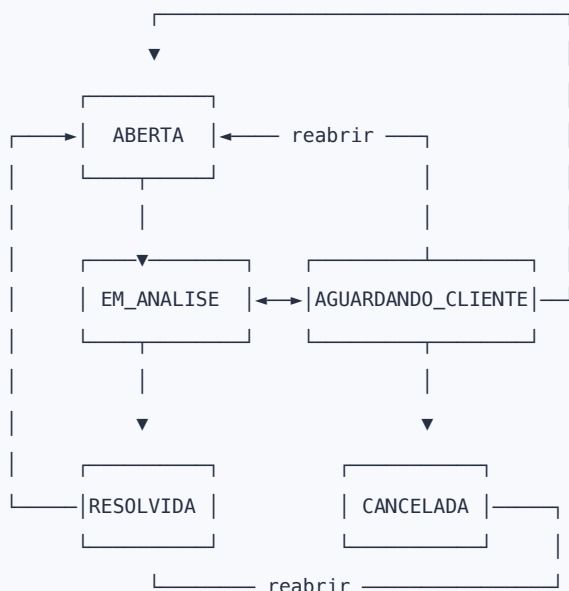
17.1 O modelo

`PendenciaCompliance`: `clienteId`, `motivo`, `criticidade`, `titulo`, `observacoes`, `prazo`, `status`, `responsavelId`, `resolvidaEm`.

`criticidade` **reutiliza** o enum `IncidenteCriticidade` (BAIXA · MEDIA · ALTA · CRITICA) em vez de criar outra escala idêntica.

Motivos: Atualização cadastral · Explicação da movimentação · Explicação sobre denúncia · Regularização do CNPJ/CPF · Outro.

17.2 Máquina de estados



`transicaoValida(de, para)` impõe as regras. Os dois estados terminais só voltam por **reabertura explícita** para `ABERTA` — `RESOLVIDA` não vai direto para `EM_ANALISE`.

17.3 Prazo

`estaVencida(prazo, status)` — vencida é a que passou do prazo **sem ter sido encerrada**. Pendência resolvida não vence. Sem prazo não vence.

`diasParaPrazo()` conta para frente e para trás, e a tela destaca em vermelho o que passou.

17.4 Fluxo operacional

```
Compliance identifica algo
  ▼
Abre pendência: cliente · motivo · criticidade · prazo · responsável
  ▼
├ Auditoria: ABRIU_PENDENCIA_COMPLIANCE
├ PendenciaEvento: statusPara = ABERTA
└ Notificação para o responsável (se não for ele mesmo)
  ▼
Responsável acompanha — cada mudança grava PendenciaEvento
  ▼
Tela de detalhe lista os DOCUMENTOS do cliente à mão
  (resolver uma pendência quase sempre passa por olhar o que já foi enviado)
  ▼
RESOLVIDA ou CANCELADA → resolvidaEm preenchido
```

17.5 Histórico

`PendenciaEvento` guarda `statusDe`, `statusPara`, `comentario`, `userId` e `createdAt`. Toda alteração — inclusive comentário sem mudança de status — vira um evento. A timeline é montada dele, não da `Auditoria`, pela mesma razão de §4.2.

17.6 O problema resolvido

Exigência regulatória tem prazo e responsável. Quando isso vive em e-mail, ninguém sabe o que está aberto, com quem e há quantos dias. O módulo dá a lista única, o estado e o vencimento — e deixa rastro de cada mudança.

18 Automações

18.1 A arquitetura no-code

```
QUANDO (gatilho) → [E condição] → ENTÃO (ação)
```

Deliberadamente pequeno: **4 gatilhos, 4 ações**, condições simples. Não é um motor genérico de workflow.

Gatilhos: `CARD_CRIADO` · `ETAPA_CONCLUIDA` · `CARD_TRANSFERIDO` · `FORMULARIO_ENVIADO`

Ações: `TRANSFERIR_FUNIL` · `NOTIFICAR` · `CRIAR_TAREFA` · `ABRIR_PENDENCIA`

18.2 Escopo e condição

`Automacao.funilId` e `etapaId` limitam o gatilho. **Nulo = qualquer** — o padrão é valer para tudo, em vez de exigir uma regra por funil.

`condicao` é JSONB no formato `{ "todas": [ {campo, operador, valor} ] }`. `todas` é **E lógico**.

- **Campos:** `valor`, `probabilidade`, `segmento`, `operacao`, `titulo`
- **Operadores:** `eq`, `ne`, `gt`, `gte`, `lt`, `lte`, `contem`

Condição ausente ou vazia **passa**. Condição malformada **não dispara** — falha fechada, e há teste para isso.

18.3 Por que não existe event bus

O ponto único já existia:

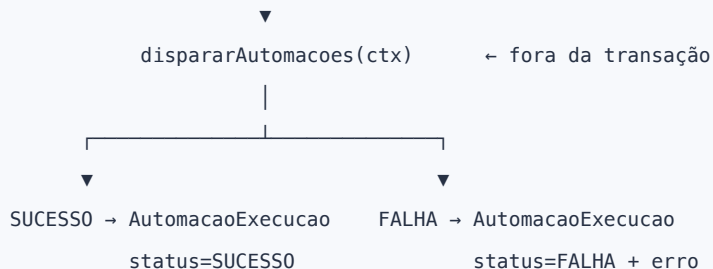
```
registrarMovimentacao() ← lib/pipeline-db.ts

  ▲           ▲           ▲           ▲
  |           |           |           |
POST /deals  /mover      /transferir  PATCH /etapas/[id]
CRIACAO     MOVIMENTO_ETAPA  TRANSFERENCIA  (realocação – sistêmica,
                                           não dispara)
```

Toda mudança de card passa por aí. Um event bus acrescentaria fila, entrega garantida, ordenação e observabilidade própria para resolver o que quatro chamadas já resolvem.

18.4 Por que roda depois da transação

TRANSAÇÃO { Deal.update + PipelineMovimentacao.create } — COMMIT



Uma automação com defeito não pode desfazer um movimento de card que já aconteceu.

`dispararAutomacoes` nunca lança — nem quando a própria listagem de automações falha. Toda falha vira linha de log.

A execução é **sequencial de propósito**: duas automações que mexem no mesmo card precisam enxergar o efeito uma da outra.

18.5 `AutomacaoExecucao`

`automacaoId`, `status` (SUCESSO/FALHA/IGNORADA), `dealId`, `respostaId`, `contexto` (JSONB), `resultado`, `erro`, `createdAt`.

É onde se descobre **por que algo não rodou**. A tela mostra as 50 últimas execuções ao clicar no contador.

18.6 As ações, em detalhe

Ação	O que faz	Reaproveita
TRANSFERIR_FUNIL	move o card, grava movimentação, audita	serviço da v11
NOTIFICAR	resolve destinatários (usuário ou todos de uma role) e cria notificações	Notificacao
CRIAR_TAREFA	cria Tarefa e notifica o responsável	Tarefa (já existia)
ABRIR_PENDENCIA	cria pendência + evento, notifica. Exige cliente no card	PendenciaCompliance

`interpolador()` substitui `{{card}}`, `{{cliente}}`, `{{funil}}` e `{{etapa}}` nos textos configurados; variável ausente vira —.

18.7 Validação na gravação

`validarConfiguracao()` impede salvar uma automação num estado que só falharia na hora de executar: transferir sem destino, notificar sem destinatário ou sem título, tarefa/pendência sem responsável.

19 Formulários

19.1 Os cinco modelos

Modelo	Papel
Formulario	nome, descrição, ativo, criador
FormularioVersao	versao (int), definicao (JSONB), publicadaEm
FormularioLink	token, cliente opcional, expiração, usosMax, revogação
FormularioResposta	valores (JSONB), aceite, assinatura, IP, status
FormularioAnexo	ponte 1:1 entre resposta e Documento

19.2 Os 24 tipos de campo

Categoria	Tipos	Exemplo de uso
Texto	TEXTO , TEXTO_LONGO	nome fantasia, justificativa
Numérico	NUMERO , MOEDA , PERCENTUAL , VOLUMETRIA	ticket médio, taxa, volume esperado
Data	DATA , DATA_HORA	data de constituição, agendamento
Contato	EMAIL , TELEFONE	contato do responsável
Documento	DOCUMENTO	CNPJ ou CPF (valida 11 ou 14 dígitos)
Escolha	SELECT , MULTIPLA , CHECKBOX , RADIO , DROPDOWN	porte, canais de operação
Arquivo	UPLOAD , UPLOAD_MULTIPLO	contrato social, comprovantes
Jurídico	ACEITE , ASSINATURA	termos de uso, assinatura digitada
Estrutura	SECAO , TEXTO_INFORMATIVO , IMAGEM	dividir e explicar o formulário
Técnico	OCULTO	valor fixo não exibido

Os três de **estrutura** são *decorativos*: não coletam valor e não contam como pergunta. Um formulário só com eles é recusado na validação.

19.3 Propriedades por campo

obrigatorio · placeholder · ajuda (help text) · opcoes · valorPadrao · colunas (12 / 6 / 4 do grid) · min / max · conteudo (texto informativo, descrição de seção ou URL da imagem).

19.4 O construtor

`/dashboard/formularios/[id]` , três abas:

Campos — paleta com os 24 tipos à esquerda, canvas ao centro, painel de propriedades à direita. **Drag & drop** para reordenar dentro da seção, mais botões ↑↓ (mais previsíveis e acessíveis que arrastar sozinho). Seções podem ser criadas e removidas.

A prévia usa **o mesmo componente** que a página pública (`RenderCampo`) — o que o administrador vê montando é literalmente o que o respondente verá.

Aparência — capa, logo, título, subtítulo, introdução, rodapé e mensagem de sucesso.

Links externos — gerar, copiar, revogar e reativar.

19.5 Versionamento e imutabilidade

```
PUT /api/formularios/versoes/[id]
|
├ versão SEM respostas → atualiza a própria versão
|
└ versão COM respostas → cria a versão SEGUINTE
                        (a atual fica congelada)
```

Por que: uma resposta aponta para a **versão**, não para o formulário. Sem isso, editar uma pergunta hoje mudaria o significado do que foi respondido ontem. O construtor avisa na tela quando a versão está congelada.

19.6 O link público `/f/[token]`

É a **única rota sem sessão** do sistema, e por isso:

- localiza pelo **token aleatório de 24 caracteres**, nunca por id — id em URL pública vaza a existência e a ordem dos registros;
- devolve **apenas a definição e a aparência**, jamais respostas de terceiros;
- **revalida o link a cada acesso**: revogado, expirado, esgotado;
- a página tem `robots: { index: false, follow: false }`.

`linkUtilizavel()` devolve `REVOGADO` , `EXPIRADO` , `ESGOTADO` ou `null` . Cada caso vira uma mensagem específica com HTTP 410.

`/f/` e `/api/formularios/publico/` estão em `PUBLIC_PATHS` no proxy.

19.7 Validação da resposta

`validarResposta(definicao, valores)` roda **no navegador** (retorno imediato) e **no servidor** (a que vale) — mesma função, sem duplicar regra.

Valida obrigatoriedade, e-mail, telefone (mín. 10 dígitos), CNPJ/CPF (11 ou 14 dígitos), faixa numérica, percentual entre 0 e 100, volumetria não negativa, data válida e opção pertencente à lista.

Campos de upload são **pulados** nessa função — viajam como arquivos, fora de `valores`, e são validados por `prepararAnexos`.

19.8 Anexos

Quando há arquivos, a resposta vai como **multipart**: o JSON em `dados` e cada arquivo em `anexo:<campoId>`.

`prepararAnexos()` valida antes de qualquer byte subir:

- **link sem cliente vinculado não aceita anexo** — um `Documento` pertence a um `Cliente`, e arquivo sem cliente não teria onde morar. A resposta **sem** anexo continua funcionando normalmente;
- campo obrigatório exige arquivo;
- `UPLOAD` aceita um só; `UPLOAD_MULTIPLO` aceita vários;
- extensão, MIME e tamanho pelas mesmas regras dos Documentos;
- arquivo para campo inexistente é tratado como payload forjado, não engano.

Os bytes sobem **antes** da transação (storage não participa de transação de banco); se a gravação falhar depois, o que subiu é removido.

Cada anexo vira um `Documento` com `origem = FORMULARIO`, categoria `OUTROS`, e um `FormularioAnexo` ligando à resposta. O autor do registro é quem criou o link — a resposta pública não tem sessão.

19.9 Painel

`/dashboard/formularios` mostra links gerados, respostas, **taxa de resposta** (respostas ÷ links) e **taxa de conclusão** (concluídas ÷ respostas), além do quadro por status.

20 Auditoria

20.1 O modelo

`Auditoria` : `acao` , `entidade` , `entidadeId` , `detalhes` , `userId` , `createdAt` . Gravada por `logAudit()` (`lib/audit.ts`), que **engole erros** — auditar nunca pode derrubar a operação auditada.

Consulta em `/dashboard/auditoria` , restrita ao módulo ADMIN.

20.2 O que é auditado hoje

Domínio	Ações
Certificados	<code>CRIOU_VERSAO_CERTIFICADO</code> , <code>EDITOU_VERSAO_CERTIFICADO</code> , <code>ENVIOU_CERTIFICADOS</code> , <code>CANCELOU_ENVIO_CERTIFICADOS</code> , <code>REVELOU_SENHA_CERTIFICADO</code> , <code>NEGOU_REVELACAO_SENHA</code>
Documentos	<code>ENVIOU_DOCUMENTO</code> , <code>BAIXOU_DOCUMENTO</code> , <code>INATIVOU_DOCUMENTO</code> , <code>REATIVOU_DOCUMENTO</code>
Compliance	<code>ABRIU_PENDENCIA_COMPLIANCE</code> , <code>ATUALIZOU_PENDENCIA_COMPLIANCE</code>
Pipeline	<code>CRIOU_FUNIL</code> , <code>EDITOU_FUNIL</code> , <code>INATIVOU_FUNIL</code> , <code>REATIVOU_FUNIL</code> , <code>REORDENOU_FUNIS</code> , <code>CRIOU_ETAPA</code> , <code>EDITOU_ETAPA</code> , <code>INATIVOU_ETAPA</code> , <code>REATIVOU_ETAPA</code> , <code>REORDENOU_ETAPAS</code> , <code>DEFiniu_PERMISSOES_FUNIL</code> , <code>MOVEU_CARD</code> , <code>TRANSFERIU_CARD</code>
Automações	<code>CRIOU_AUTOMACAO</code> , <code>EDITOU_AUTOMACAO</code> , <code>ATIVOU_AUTOMACAO</code> , <code>DESATIVOU_AUTOMACAO</code> , <code>AUTOMACAO_TRANSFERIU_CARD</code>
Formulários	<code>CRIOU_FORMULARIO</code> , <code>EDITOU_FORMULARIO</code> , <code>SALVOU_VERSAO_FORMULARIO</code> , <code>PUBLICOU_VERSAO_FORMULARIO</code> , <code>CRIOU_VERSAO_FORMULARIO</code> , <code>GEROU_LINK_FORMULARIO</code> , <code>REVOGOU_LINK_FORMULARIO</code> , <code>REATIVOU_LINK_FORMULARIO</code>
Volumetria	<code>CRIOU_VOLUMETRIA</code> , <code>EDITOU_VOLUMETRIA</code> , <code>INATIVOU_VOLUMETRIA</code> , <code>REATIVOU_VOLUMETRIA</code>
Carteira	<code>CRIOU_CLIENTE</code> , <code>EXCLUIU_CLIENTE</code> , <code>REGISTROU_MOVIMENTO_DIARIO</code>

20.3 Os dois casos mais sensíveis

Revelação de senha de certificado. A auditoria é gravada **antes** de a resposta sair. Se a gravação falhar, a senha não é entregue. A tentativa **negada** também é registrada — quem tentou ver o que não pode é informação de segurança.

Download de documento. Cada emissão de signed URL fica registrada com cliente e nome do arquivo.

20.4 O que NÃO é registrado

Senhas, chaves, tokens e conteúdo de arquivo. `detalhes` é texto curto e descritivo — nome do cliente, referência do certificado, transição de status.

20.5 Três tabelas de histórico coexistem com a Auditoria

`PipelineMovimentacao`, `PendenciaEvento` e `AutomacaoExecucao` existem porque `Auditoria.detalhes` é texto livre e não permite montar timeline com origem e destino, nem filtrar. Cada uma serve a uma tela específica; a `Auditoria` serve à trilha global.

21 Segurança

21.1 Autenticação

JWT próprio assinado com `JWT_SECRET` (HS256, `jsonwebtoken`), entregue em cookie:

```
httpOnly : true      - JavaScript não lê
secure   : em produção
sameSite : lax
maxAge   : 7 dias
path     : /
```

Senhas com `bcryptjs`. O login **não distingue** "usuário não existe" de "senha errada" — ambos respondem `Credenciais inválidas` com 401, para não permitir enumeração de usuários.

Usuário inativo (`active: false`) não autentica.

21.2 Autorização em três camadas

Camada	O que faz	O que não faz
Proxy	autentica; bloqueia módulo desligado; gate por role de módulo	não separa sub-rotas (casa por prefixo)
Página	<code>redirect()</code> server-side em 8 telas administrativas	—
API	<code>hasPermission</code> / <code>acessoAoFunil</code> em toda escrita	—

O frontend esconde botões por conveniência. **A decisão é sempre do servidor.**

21.3 Segredos

Sete variáveis, nenhuma no repositório:

Variável	Exposição
<code>DATABASE_URL</code>	server-only
<code>DIRECT_URL</code>	server-only
<code>JWT_SECRET</code>	server-only
<code>SUPABASE_SERVICE_ROLE_KEY</code>	server-only, jamais <code>NEXT_PUBLIC_</code>
<code>CERTIFICADO_ENCRYPTION_KEY</code>	server-only
<code>NEXT_PUBLIC_SUPABASE_URL</code>	pública por natureza
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	pública por desenho (protegida por RLS)

Garantia estrutural verificada: **nenhum arquivo** `'use client'` importa `lib/storage`, `lib/crypto-certificado`, `lib/prisma` ou `lib/auth`. Foi por isso que `lib/arquivos.ts` foi extraído — o construtor de formulários precisa dos validadores no navegador sem arrastar o SDK do Supabase junto.

`.env.example` lista os nomes, nunca valores. `.gitignore` cobre `.env*` com exceção do exemplo.

21.4 Storage

- Bucket `cliente-arquivos` com `public = false` — **verificado em Production**;
- URL pública devolve **400**;
- signed URL de 60 s funciona e **expira** (verificado: 200 → 400 após o prazo);
- upload server-side com validação tripla (extensão + MIME + tamanho);
- `nomeSeguro()` impede travessia de diretório; a chave sempre fica sob `clientes/{clienteId}/`.

21.5 Criptografia

AES-256-GCM para senhas de certificado, com IV aleatório por operação e authentication tag. Ver §16.6.

21.6 Link público de formulário

A única superfície sem sessão. Mitigações: token aleatório (não id), expiração opcional, limite de usos, revogação, `noindex`, revalidação a cada acesso, e resposta que devolve apenas a definição — nunca dados de terceiros.

21.7 Exposição de dados

- Notificações filtradas por `destinatarioId` no `where`, não por checagem posterior;
- `senhaCifrada` fora de todo `select` de listagem;
- `apenasProprios` aplicado no `where` da consulta do quadro, não na renderização;
- CRM restrito aos funis visíveis ao usuário.

21.8 Limitação conhecida

Quatro rotas anteriores ao trabalho recente — `/api/financeiro`, `/api/financeiro/[id]`, `/api/followup/[id]`, `/api/tarefas/[id]` — **autenticam mas não verificam permissão granular**, delegando ao gate de módulo do proxy. Registrado em §26 como débito de prioridade MÉDIA.

22 Design System

22.1 Filosofia

Três princípios, visíveis no código:

1. **Um bloco visual = um contorno.** Sem sombra em repouso; a separação vem do fio (`border-line`), não da elevação.
2. **Cor comunica status, nunca categoria.** Segmento, modelo operacional e estágio de funil usam tom neutro. Só atingimento e severidade coloriem.
3. **Ausência de dado é estado de primeira classe.** `EmptyState` e `NoData` existem para não imprimir zeros falsos.

22.2 Tipografia

Duas famílias, via `next/font/google` :

- **Inter Tight** — títulos, KPIs e **todo número financeiro** (variante estreita);
- **Inter** — corpo, tabelas, labels, navegação e formulários.

Escala em classes utilitárias: `.t-hero` , `.t-h1` , `.t-h2` , `.t-h3` , `.t-body` , `.t-sm` , `.t-label` , `.t-mono` , `.t-num` , `.t-figure` .

`.t-num` aplica tabular numbers — colunas de milhar empilham.

22.3 Tokens

Definidos em `app/globals.css` :

Superfície e texto: `--color-ink` , `--color-surface` , `--color-surface-2` , `--color-line` , `--color-line-2` , `--color-fg` , `--color-muted` , `--color-subtle`

Acento: `--color-accent` , `--color-accent-dark` , `--color-accent-soft` , `--color-on-accent` , `--bp-accent-wash`

Semânticos: `--color-pos` , `--color-warn` , `--color-neg` , `--color-alert`

Gráficos: `--bp-chart-1/2/3` , `--bp-chart-axis` , `--bp-chart-grid` , `--bp-chart-cursor` , `--bp-chart-tip-bg`

Movimento: `--bp-ease` , `--bp-t-fast` , `--bp-t-med` , `--ease-bp`

Elevação: `--bp-shadow-sm` , `--bp-shadow-hover` , `--bp-shadow-overlay`

22.4 Light / Dark Mode

`components/theme/ThemeProvider.tsx` .

A fonte da verdade é `<html data-theme>` , escrito por um **script inline antes da primeira pintura** — é o que evita o flash de tema errado. O React lê de lá com `useSyncExternalStore` , e não sincroniza por efeito: sem `setState` em

effect, sem render em cascata.

A preferência persiste em `localStorage` sob `bp-theme`. A classe `.bp-theme-ready` só é adicionada após montar, para que o primeiro paint não anime do tema errado para o certo.

`ThemeToggle` alterna. Padrão: **dark**.

22.5 Componentes

Componente	Papel
<code>Panel</code> / <code>PanelHeader</code>	superfície base; um bloco = um contorno
<code>Badge</code> / <code>Dot</code>	status com tom semântico; losango de 5px como marcador
<code>Button</code>	variantes <code>primary</code> , <code>ghost</code> , <code>subtle</code> , <code>danger</code>
<code>DataTable</code>	<code>TableShell</code> (scroll horizontal obrigatório), <code>Table</code> , <code>THead</code> sticky, <code>Th</code> , <code>Row</code> , <code>Td</code> , <code>TdTrunc</code> , <code>EmptyRow</code>
<code>StatTile</code> / <code>MetaBar</code>	cartão de KPI com sparkline e barra de meta
<code>EmptyState</code> / <code>NoData</code>	ausência de dado como estado desenhado
<code>Skeleton</code>	carregamento
<code>HairlineGrid</code>	grade de fios para blocos de indicador
<code>Figure</code>	formatação de número financeiro
<code>ChartTooltip</code>	tooltip de gráfico com os tokens do tema
<code>PageHeader</code>	cabeçalho único de página
<code>Sidebar</code> / <code>Topbar</code>	navegação derivada de <code>lib/modules.ts</code>
<code>BrandMark</code> / <code>BrandLockup</code>	integração da marca

`TableShell` existe porque 6 das 11 tabelas do app não tinham scroll horizontal e estouravam o viewport.

22.6 Formulários e responsividade

Classes `.bp-field` e `.bp-field-label` padronizam entradas. `.bp-btn-primary` padroniza a ação principal.

Layout mobile-first: sidebar vira drawer abaixo de `lg`, tabelas rolam dentro do próprio contêiner, e o corpo da página nunca rola lateralmente.

22.7 Marca

A integração atual da marca se dá por `BrandMark` / `BrandLockup` no Sidebar e na tela de login, e pelo `<title>` "**Bass Pago · RevOps**". Esta documentação não trata de especificação de marca — apenas registra como ela está integrada.

23 Testes

23.1 Estratégia

104 testes, 0 falhas, 0 pulados. Runner: `node:test` executado por `tsx`, sem framework externo e sem dependência nova (`npm test`).

O princípio: **testar a regra, não a consulta**. As funções de domínio foram separadas do acesso a dados exatamente para isso — nenhum teste precisa de banco, e a suíte roda em menos de 400 ms.

23.2 Cobertura por área

Suíte	Testes	O que garante
<code>formularios.test.ts</code>	22	definição, validação por tipo, token, ciclo do link, anexos
<code>pipeline.test.ts</code>	20	alçada, reordenação, transferência, movimento, inativação
<code>certificados.test.ts</code>	13	estoque 50, numeração 1–50, único/lote, AES-256-GCM
<code>automacoes.test.ts</code>	12	casamento de gatilho, condições, configuração das ações
<code>volumetria.test.ts</code>	11	vigência, status, sobreposição, consolidação
<code>crm.test.ts</code>	10	tempo por etapa, conversão, ciclo, gargalos
<code>storage.test.ts</code>	7	formatos, MIME × extensão, tamanho, travessia
<code>compliance.test.ts</code>	5	máquina de estados, prazo
<code>carteira.test.ts</code>	4	janela UTC, três estados do dia

23.3 Os testes que pegam o que revisão de código não pega

Alguns merecem destaque porque protegem decisões sutis:

- **apenasProprios** só restringe quando todas as regras pedem — uma concessão nominal mais ampla levanta a restrição da role.
- **A regra específica de funil vale sem a chave global** — sem esse teste, a alçada por funil poderia ser "corrigida" no futuro e quebrar o Onboarding.
- **Cifrar duas vezes a mesma senha dá resultados diferentes** — protege o IV aleatório.
- **Texto cifrado adulterado falha** — protege a authentication tag do GCM.
- **A etapa atual conta até agora** — sem isso, a etapa onde tudo empaca pareceria a mais rápida do funil.
- **Ausência de registro diário não é "não"** — protege os três estados.
- **1–10** existe em duas versões diferentes — protege a numeração relativa.
- **Link sem cliente recusa anexo mas aceita resposta** — protege a fronteira.

23.4 O que a suíte NÃO cobre

Honestamente: não há testes de integração HTTP, nem de componentes React, nem end-to-end de navegador. A validação dessas camadas foi feita por `tsc`, `build`, `lint` e verificação manual contra o banco real. Registrado em §26.

24 Deploy e infraestrutura

24.1 Ambientes

	Production	Preview
Projeto Supabase	gyvs...zchd	oikg...dgmd
PostgreSQL	17.6	17.11
Tabelas	41	35
Bucket	cliente-arquivos (privado)	não criado
URL	revenueops-cockpit.vercel.app	URLs por deployment

24.2 O processo

```
desenvolvimento local
  | npm test · tsc --noEmit · npm run lint · npm run build
  ▼
commit (branch claude/setup-deploy-cockpit-SjSte)
  |
  ▼
push → GitHub (joaolima-max/revenueops-cockpit)
  |
  |→ Vercel cria Preview automaticamente
  |
  ▼
migrations aplicadas no banco ANTES do deploy de código
  | psql "$DIRECT_URL" -f supabase-migration-vN.sql
  | prisma migrate diff → drift check
  ▼
vercel deploy --prod
  |
  ▼
vercel promote <deployment> ← necessário se houve rollback antes
  |
  ▼
smoke test
```

Armadilha real, já encontrada: um `rollback` manual **fixa o alias** de produção. Um `vercel deploy --prod` posterior cria o deployment mas **não** assume o alias — é preciso `vercel promote`. Sem isso, o domínio continua servindo o código antigo enquanto o painel mostra "Ready".

24.3 Migrations

O projeto **não usa Prisma Migrate**. Usa SQL versionado à mão (`supabase-migration-vN.sql`), convenção mantida desde a v1. `prisma migrate status` reporta "não gerenciado" — é esperado.

O comando que decide se banco e código concordam:

```
npx prisma migrate diff --from-config-datasource \  
  --to-schema prisma/schema.prisma --script
```

Saída vazia é o resultado desejado.

Regras que todas as migrations seguem:

- somente `CREATE` , `ADD` , `INSERT` e `UPDATE` restrito;
- **zero** `DROP TABLE` , `DROP COLUMN` , `DELETE FROM` , `TRUNCATE` ;
- idempotentes: `IF NOT EXISTS` , `DO $$ ... duplicate_object` , `ON CONFLICT DO NOTHING` ;
- ids de seed fixos e legíveis (`fnl_vendas` , `etp_vnd_prospeccao`);
- SQL e `schema.prisma` no mesmo commit.

24.4 Conexões

Duas, com papéis distintos:

- `DATABASE_URL` — transaction pooler, porta 6543. É o que a aplicação usa em runtime.
- `DIRECT_URL` — conexão direta, porta 5432. É por onde as migrations rodam: DDL precisa de sessão real, e o pooler pode cortar no meio.

24.5 Storage

Criado por `npm run setup:storage` (`scripts/setup-storage.ts`), idempotente: cria o bucket se não existir e **fecha** um bucket que esteja público.

24.6 Estado de Production no momento deste documento

Deployment	<code>dpl_2QivpGp2yog76yEpEwhVwcSvhz2A</code>
Commit	<code>4890630</code>
Banco	41 tabelas · 31 enums · 67 FKs · 100 índices
Dados	3 usuários · 10 parâmetros · 9 registros de auditoria · 1 lançamento diário
Drift	zero nos 35 modelos do schema
Bucket	criado, privado, validado

25 Manutenção — guia para quem vier depois

25.1 Alterar o schema

1. edite `prisma/schema.prisma`
2. crie `supabase-migration-v<N+1>.sql` com o SQL equivalente
3. `npx prisma generate`
4. aplique num banco de desenvolvimento e rode o drift check
5. `npx tsc --noEmit && npm test && npm run build`
6. commit: `schema.prisma` E o `.sql` juntos, sempre

Regras que não podem ser quebradas:

- nunca `DROP TABLE`, `DROP COLUMN`, `DELETE FROM` ou `TRUNCATE` numa migration;
- toda migration é idempotente;
- `updatedAt` **não** leva `DEFAULT` no SQL — Prisma trata `@updatedAt` no cliente, e um default cria drift (já aconteceu; ver §3.7);
- relação opcional **sem** `onDelete` explícito assume `SetNull` no Prisma. Se o SQL usar outro comportamento, declare no schema (já aconteceu);
- se a migration semeia linhas, informe `createdAt` e `updatedAt` — sem o default, o INSERT falha (já aconteceu).

25.2 Adicionar uma permissão

1. `lib/permissions.ts` → nova entrada em `ALL_PERMISSIONS` (key, label, group)
2. `DEFAULT_PERMISSIONS` → adicione aos papéis que devem tê-la por padrão
(ADMIN recebe automaticamente: a lista dele é `ALL_PERMISSIONS.map`)
3. use `hasPermission(session.permissoes, 'sua_chave', session.role)` NA API
4. se houver página dedicada, adicione o `redirect()` server-side

Cuidado: usuários com lista explícita salva **não** recebem a chave nova. Isso costuma ser o comportamento desejado, mas precisa ser consciente.

25.3 Criar uma rota de API

- ```
app/api/<recurso>/route.ts
```
1. `getSession()` → 401
  2. `hasPermission` / `acessoAoFunil` → 403
  3. validar entrada (nunca confie no cliente)
  4. transação quando houver mais de uma escrita
  5. `logAudit()` para ação sensível
  6. efeitos colaterais (notificar, automações) **DEPOIS** do commit

**Regra do App Router:** `route.ts` só pode exportar métodos HTTP. Helper compartilhado vai para `lib/` — já quebrou o build uma vez.

## 25.4 Adicionar uma automação

Nada de código é necessário para **criar** uma automação — é tela. Para adicionar um **novo gatilho ou ação**:

1. `lib/automacoes.ts` → enum + label
2. `prisma/schema.prisma` + migration → ALTER TYPE ADD VALUE
3. `lib/automacoes-db.ts` → o case no switch de executar()
4. `validarConfiguracao()` → o que a nova ação exige
5. teste em `tests/automacoes.test.ts`

O gatilho novo precisa ser disparado de algum lugar: se for sobre card, o ponto é `registrarMovimentacao()`.

## 25.5 Adicionar um tipo de campo de formulário

1. `lib/formularios.ts` → TIPOS\_CAMPO + TIPO\_CAMPO\_LABELS
2. se coleta valor, adicione o case em `validarResposta()`
3. `components/formularios/RenderCampo.tsx` → o case de renderização
4. se tiver propriedade nova, adicione ao painel do construtor
5. teste

Não é preciso migration: a definição é JSON.

## 25.6 Adicionar um módulo

1. `lib/modules.ts` → entrada em MODULES com key, label, route, api[], roles[]
2. `components/dashboard/nav-icons.tsx` → ícone para a key
3. `app/dashboard/<rota>/page.tsx` com guarda server-side
4. `app/api/<recurso>/route.ts`
5. permissões em `lib/permissions.ts`

## 25.7 Antes de qualquer publicação

```
npx prisma generate
npm test # 104 devem passar
npx tsc --noEmit
npm run lint # confira que não há problema NOVO
npm run build
```

## 25.8 Invariantes do produto

Coisas que o sistema decidiu e que não devem ser revertidas sem decisão explícita:

1. **Uma fonte por indicador.** Nada além de `LancamentoDiario` grava TPV, receita tarifária, saldo, transações ou MEDs.
  2. **Ausência de dado é `null`, nunca zero.**
  3. **Nada é excluído** — funil, etapa, documento, envio, link e usuário são inativados.
  4. **Float é derivado.**
  5. **Não existe TPV por cliente.**
  6. **Versão de formulário respondida é imutável.**
  7. **Automação nunca derruba a operação principal.**
  8. **Autorização é server-side.**
  9. **Senha de certificado nunca aparece em listagem, log ou URL.**
  10. **O bucket nunca é público.**
-



## 26 Limitações e débitos técnicos

---

Lista honesta, apurada no código. Nada inventado.

### CRÍTICO

*Nenhum item.* O sistema está em Production com banco migrado, drift zero, storage privado validado e segredos fora do cliente.

### ALTO

**A1 — Smoke test da UI autenticada nunca foi executado.** Não há credencial disponível de nenhum dos 3 usuários (hashes bcrypt). Todas as telas autenticadas — Dashboard, Carteira, CRM, Pipeline, Notificações, Documentos, Certificados, Compliance, Automações, Formulários — foram validadas por `tsc`, `build`, testes de domínio e escrita real contra o banco, **mas nunca clicadas em Production**. Risco: erro de renderização ou de integração que só aparece com sessão.

**A2 — `CERTIFICADO_ENCRYPTION_KEY` existe apenas no Vercel.** Foi gerada e gravada diretamente nas variáveis de Production. Não há cópia em cofre. Perder o acesso ao Vercel torna **irrecuperável** toda senha de certificado já cifrada. Hoje não há certificado emitido, então o custo de rotacionar ainda é zero — essa janela não fica aberta para sempre.

### MÉDIO

**M1 — Quatro rotas antigas sem permissão granular.** `/api/financeiro`, `/api/financeiro/[id]`, `/api/followup/[id]`, `/api/tarefas/[id]` autenticam mas delegam autorização ao gate de módulo do proxy. Anteriores ao trabalho recente.

**M2 — Seis tabelas legadas no banco de Production.** `Forecast`, `ForecastGeral`, `IncidenteCliente`, `PedidoCobavel`, `Processamento`, `ReceitaRealizada` — todas vazias, sem modelo Prisma correspondente. Mais o enum `PedidoStatus`, três colunas nullable (`Cliente.volumeMinimo`, `Incidente.satisfacao`, `Meta.realizado`) e três valores extras em `MetaTipo`. Nada quebra; o `migrate diff` reporta como diferença.

**M3 — Dois componentes órfãos.** `app/dashboard/forecast/ForecastClient.tsx` referencia `/api/forecast-geral` (rota inexistente) e um modelo `ForecastGeral` que não está no schema. `app/dashboard/relatorios/RelatoriosClient.tsx` também não é importado por ninguém. Ambos anteriores ao trabalho recente; não entram no bundle.

**M4 — `Deal.stage` é dupla verdade parcial.** O enum legado convive com `etapaId`, sincronizado apenas no funil de Vendas. `app/api/relatorios` foi escopado a esse funil, mas a duplicidade permanece.

**M5 — Ausência de testes de integração e de componente.** Ver §23.4.

**M6 — `COMERCIAL` e `GESTOR` coexistem.** Quatro valores de enum para três tipos de usuário. Migrar exigiria `UPDATE` em usuários reais e ajuste no seed do funil de Vendas.

## BAIXO

**B1 — 39 avisos de lint pré-existent**s, nenhum nos arquivos recentes. Majoritariamente `no-unused-vars` e `react-hooks/set-state-in-effect` em telas antigas.

**B2 — Reordenação por botões** ↑ ↓ na administração de funis e etapas, em vez de arrastar. Mais previsível e acessível, mas menos fluido.

**B3 — Sino de notificações faz polling a cada 60 s.** Sem realtime. Adequado ao volume atual.

**B4 — Anexo de formulário exige link vinculado a cliente.** Consequência da arquitetura ( `Documento` pertence a `Cliente` ), não bug — mas limita o uso de links genéricos com upload.

**B5 — Preview sem banco próprio configurado.** A `DATABASE_URL` de Preview no Vercel não é utilizável; o projeto Supabase de Preview existe e está migrado, mas não está ligado ao ambiente.

---

## 27 Roadmap sugerido

---

Baseado no que o código mostra, sem inventar necessidade.

### Curto prazo — estabilidade e governança

1. **Executar o smoke test autenticado** (resolve A1). É a lacuna de validação mais relevante.
2. **Copiar a `CERTIFICADO_ENCRYPTION_KEY` para um cofre** (resolve A2). Enquanto não houver certificado emitido, rotacionar custa zero.
3. **Adicionar `hasPermission` às quatro rotas antigas** (M1).
4. **Remover os dois componentes órfãos** (M3) — não entram no bundle, mas confundem quem chega.
5. **Ligar um banco ao ambiente de Preview** (B5), para validar mudanças antes de Production.

### Médio prazo — produtividade e confiança

6. **Testes de integração HTTP** nas rotas sensíveis: revelação de senha, download de documento, transferência de card, resposta pública de formulário.
7. **Aposentar `Deal.stage`** (M4): migrar `app/api/relatorios` para `etapaId` e marcar a coluna como deprecada.
8. **Decidir sobre `COMERCIAL → GESTOR`** (M6).
9. **Limpar o legado do banco** (M2) — com migration dedicada e aprovação explícita, já que envolve `DROP`.
10. **Exportação de relatórios** em CSV/PDF a partir das telas de Relatórios e CRM.

### Longo prazo — analytics e escala

11. **Materializar métricas de CRM** se o volume de `PipelineMovimentacao` crescer a ponto de o cálculo em tempo real pesar. Hoje não pesa.
  12. **Ampliar as automações** com novos gatilhos (pendência resolvida, documento enviado, volumetria não atingida) mantendo o desenho sem event bus.
  13. **Relatórios sobre respostas de formulário**, o custo consciente assumido em §4.5.
  14. **Notificações em tempo real** se o polling de 60 s virar limitação.
  15. **Retenção e arquivamento** de `Auditoria` e `AutomacaoExecucao`, que crescem monotonicamente.
-

## 28 Glossário

---

| Termo                    | Significado no sistema                                                                                                        |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>TPV</b>               | <i>Total Payment Volume</i> — volume financeiro processado. Agregado da empresa, nunca por cliente                            |
| <b>MED</b>               | Mecanismo Especial de Devolução — pedido de devolução do Pix. Alto % indica problema de qualidade                             |
| <b>Float</b>             | Rendimento do saldo que dorme em conta. $\text{min}(\text{saldo D}, \text{saldo D}+1) \times \text{multiplicador}$ . Derivado |
| <b>Take Rate</b>         | $\text{receita tarifária} \div \text{TPV}$ . Eficiência de monetização                                                        |
| <b>Saldo que dorme</b>   | Menor saldo entre dois dias consecutivos — o que atravessou a noite                                                           |
| <b>Volumetria mínima</b> | Cláusula contratual de quantidade mínima de transações por mês, por cliente                                                   |
| <b>Vigência</b>          | Intervalo <code>YYYY-MM</code> em que um contrato de volumetria vale                                                          |
| <b>Pipeline</b>          | O módulo de funis configuráveis                                                                                               |
| <b>Funil</b>             | <code>PipelineFunil</code> — processo com etapas e alçadas próprias                                                           |
| <b>Etapas</b>            | <code>PipelineEtapa</code> — coluna do quadro                                                                                 |
| <b>Card</b>              | O <code>Deal</code> movendo-se pelo pipeline                                                                                  |
| <b>Transferência</b>     | Mudança de funil (≠ mover, que troca de etapa)                                                                                |
| <b>Alçada</b>            | Conjunto das 6 permissões de um funil para uma role ou usuário                                                                |
| <b>Lead</b>              | Contato comercial antes de virar cliente                                                                                      |
| <b>Cliente</b>           | Empresa contratante da Bass Pago                                                                                              |
| <b>Gestor</b>            | Responsável pela carteira ( <code>Cliente.gestorId</code> )                                                                   |
| <b>Farmer</b>            | Nome de mercado para o Gestor. Não há campo separado                                                                          |
| <b>Owner</b>             | Vínculo legado <code>Cliente.ownerId</code> , preservado                                                                      |
| <b>Compliance</b>        | Módulo de pendências regulatórias                                                                                             |
| <b>Pendência</b>         | <code>PendenciaCompliance</code> — item com motivo, prazo, responsável e status                                               |
| <b>Automação</b>         | Regra no-code: gatilho → condição → ação                                                                                      |
| <b>Gatilho</b>           | Evento que dispara uma automação                                                                                              |
| <b>Formulário</b>        | Estrutura versionada de coleta de dados                                                                                       |
| <b>Versão publicada</b>  | Snapshot imutável de um formulário                                                                                            |
| <b>Link público</b>      | <code>/f/[token]</code> — acesso sem sessão, por token aleatório                                                              |
| <b>Certificado</b>       | Unidade dentro de uma versão/ZIP, com senha cifrada                                                                           |

| Termo                       | Significado no sistema                                                                                                                                                                                           |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Versão (certificado)</b> | O ZIP: 50 certificados, estoque global                                                                                                                                                                           |
| <b>Envio</b>                | Distribuição de 1 (único) ou 10 (lote) certificados a um cliente                                                                                                                                                 |
| <b>Signed URL</b>           | Link temporário de 60 s para baixar do bucket privado                                                                                                                                                            |
| <b>Bucket privado</b>       | <code>cliente-arquivos</code> , sem leitura pública                                                                                                                                                              |
| <b>Drift</b>                | Divergência entre <code>schema.prisma</code> e o banco real                                                                                                                                                      |
| <b>Alerta</b>               | Verificação automática do período, sem dono e sem leitura                                                                                                                                                        |
| <b>Notificação</b>          | Mensagem para um usuário, com lida/não lida                                                                                                                                                                      |
| <b>Lançamento diário</b>    | <code>LancamentoDiario</code> — fonte oficial dos 5 indicadores                                                                                                                                                  |
| <b>Movimento diário</b>     | Indicador binário   por cliente-dia. Não é TPV |

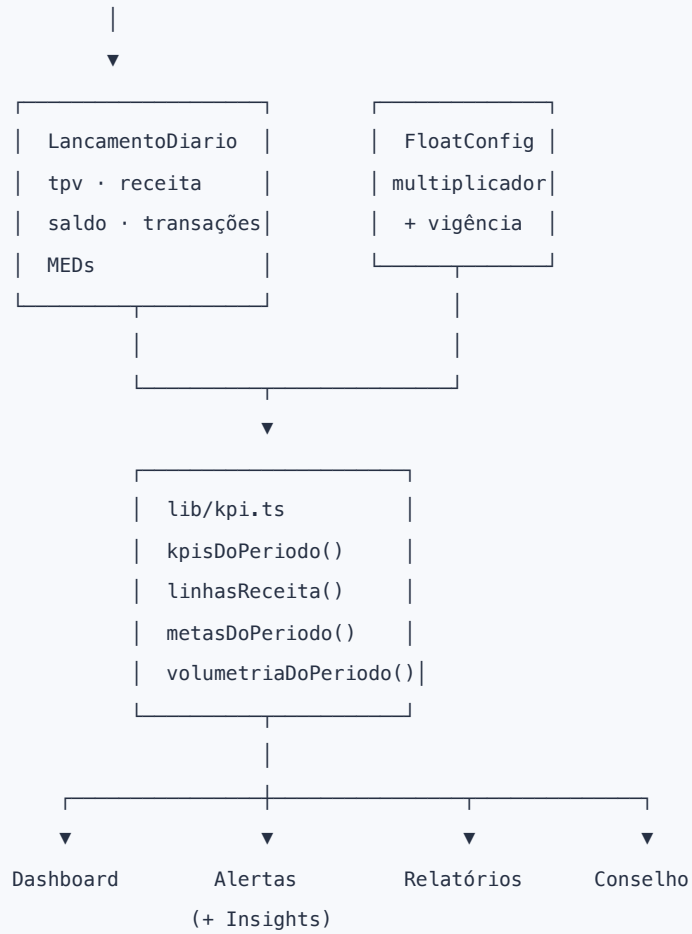
## 29 Diagramas consolidados

### 29.1 Arquitetura técnica



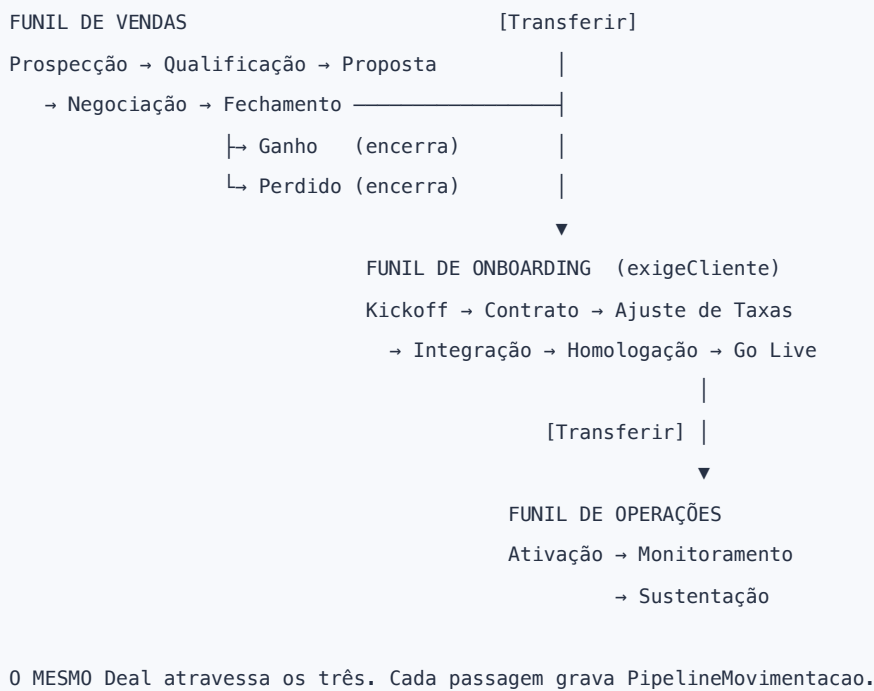
## 29.2 Fluxo de dados dos indicadores

entrada manual diária

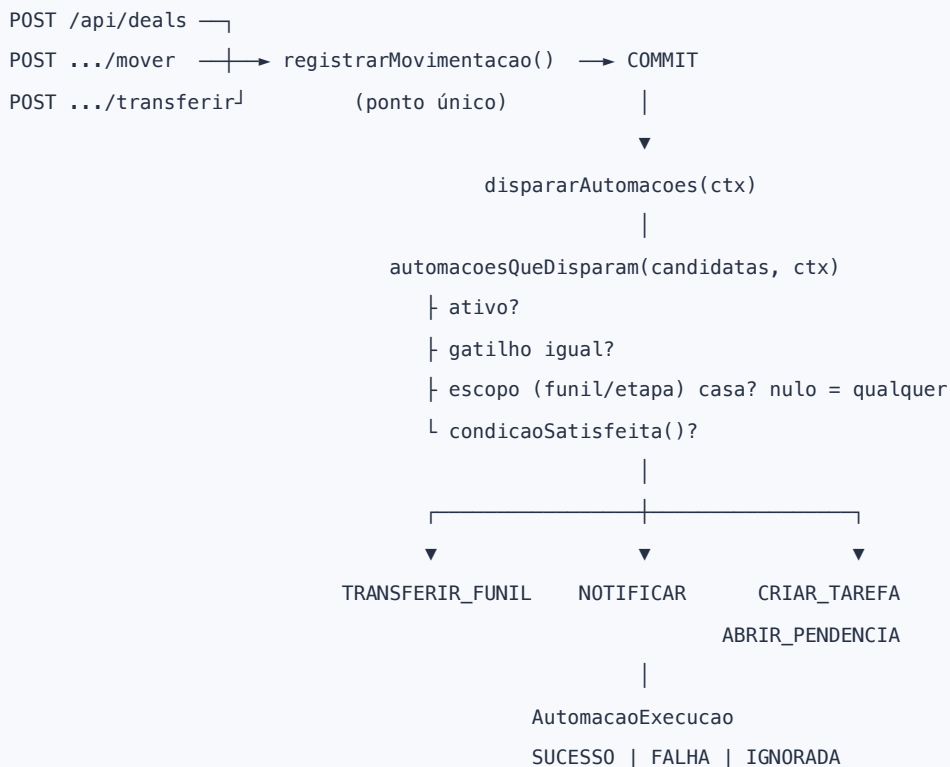




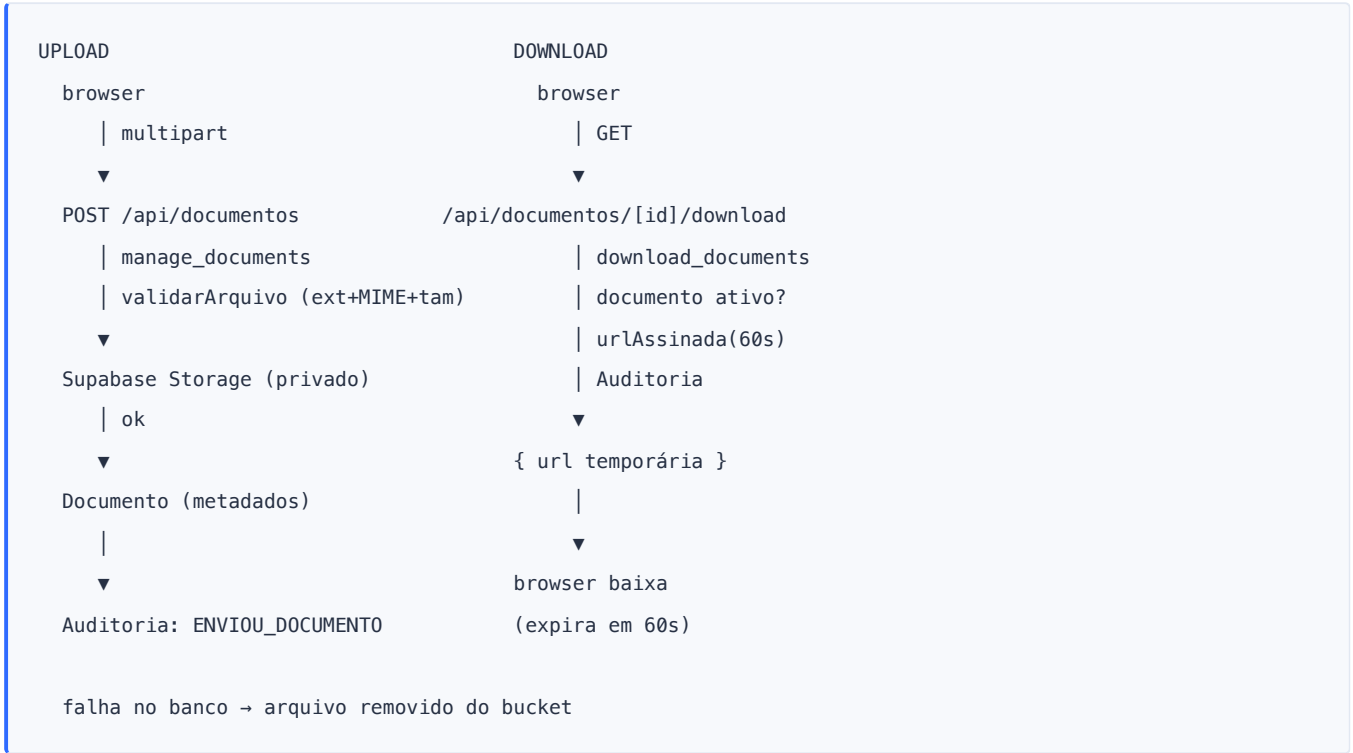
## 29.3 Pipeline e transferência



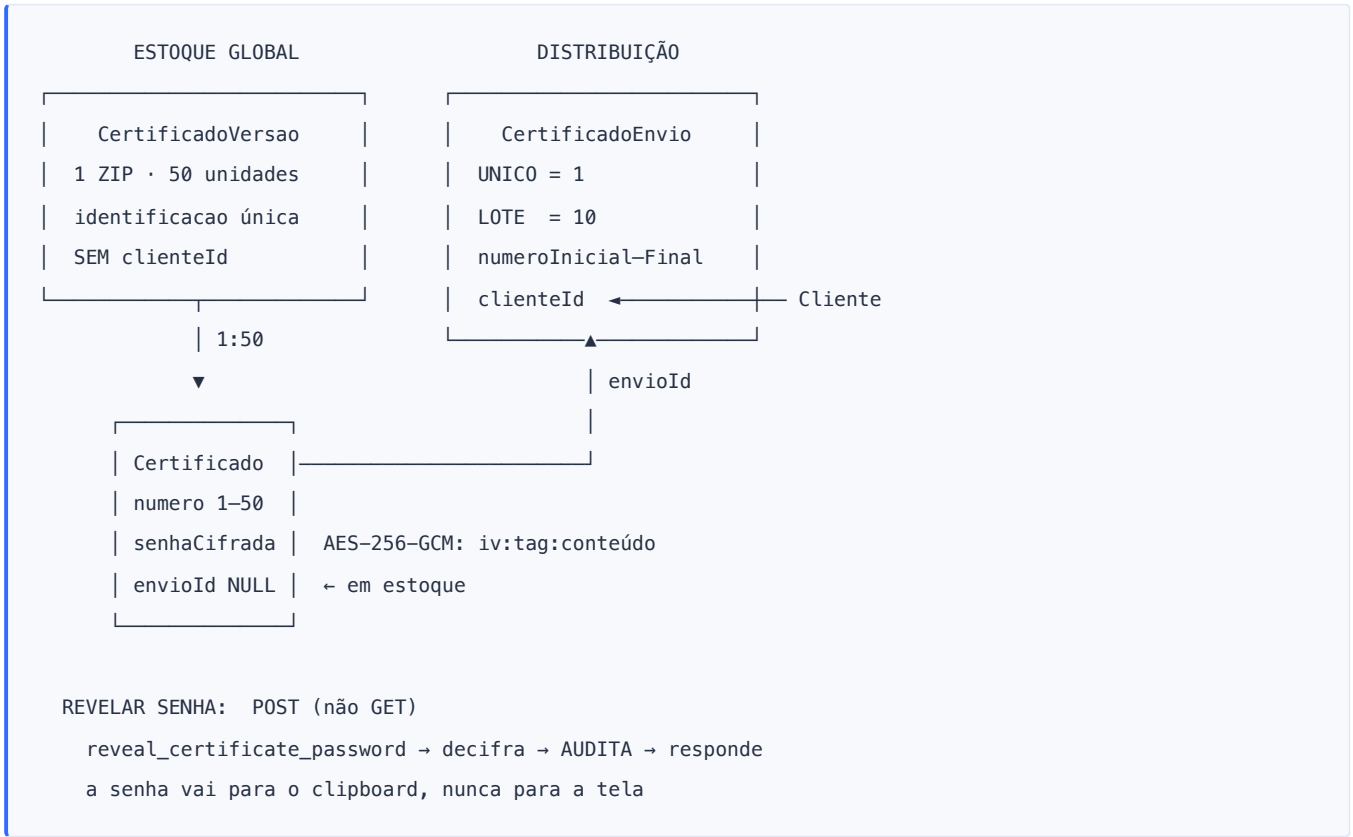
## 29.4 Automações



29.5 Documentos e Storage



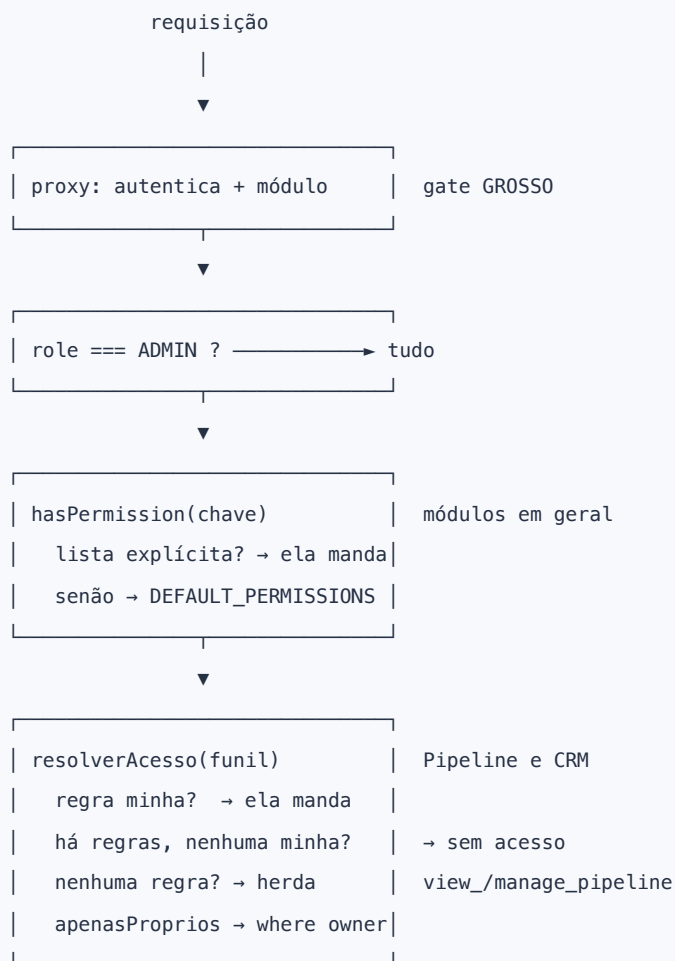
29.6 Certificados



## 29.7 Formulários



## 29.8 Permissões



Documento gerado a partir do commit [4890630](#) em 10 de setembro de 2026. Reflete o código real do repositório e o estado verificado do banco de Production. Nenhuma credencial, chave, token ou connection string está contida neste documento.